DAYANANDA SAGAR UNIVERSITY

SCHOOL OF ENGINEERING

MACHINE LEARNING

School of Engineering			
Department of Computer Science and Engineering			
Course Code:		Semester:	V
Course Title:	Machine Learning	Academic Year:	2026 - 2027
Faculty Name:	Prof.Shilpa Sudheendran		

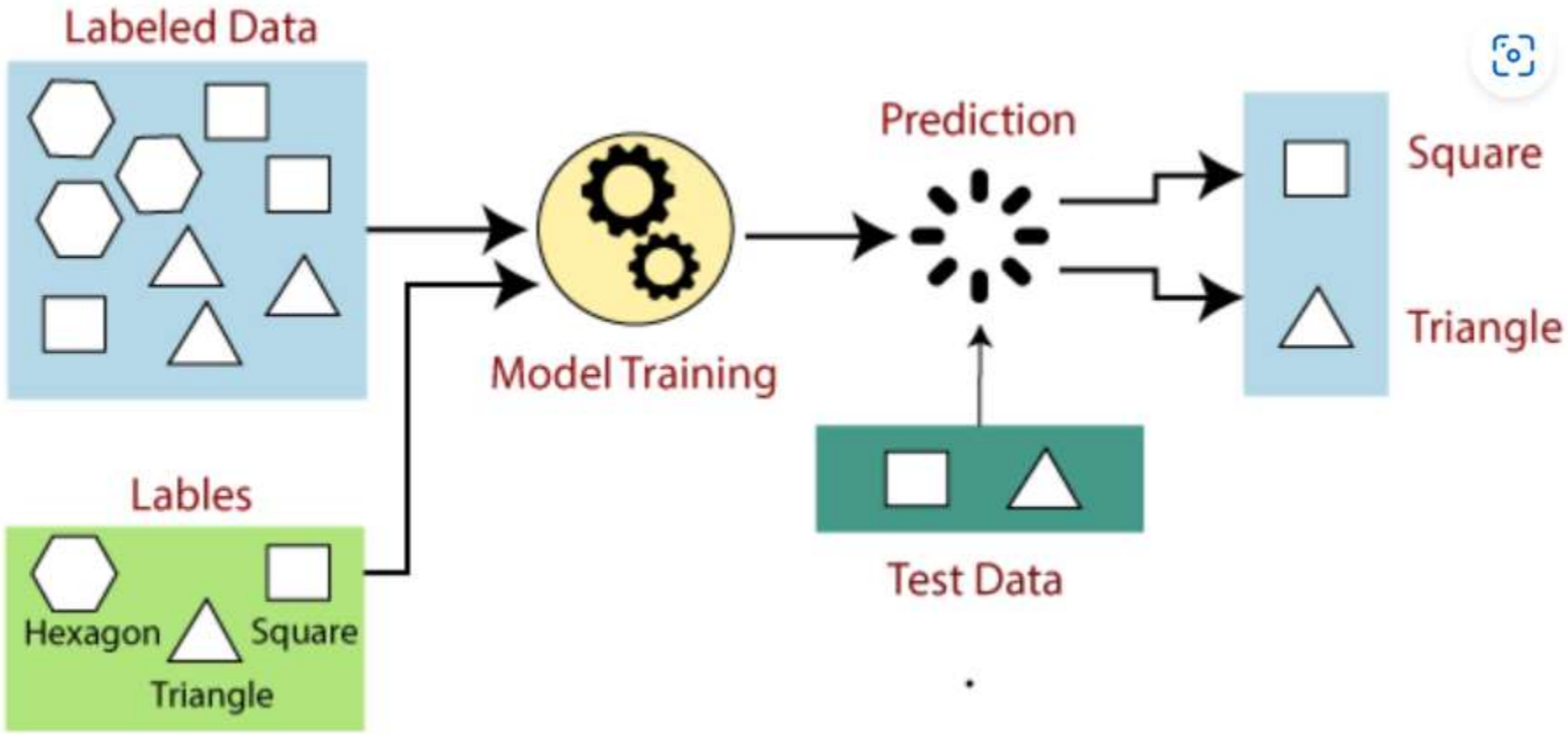
UNIT II

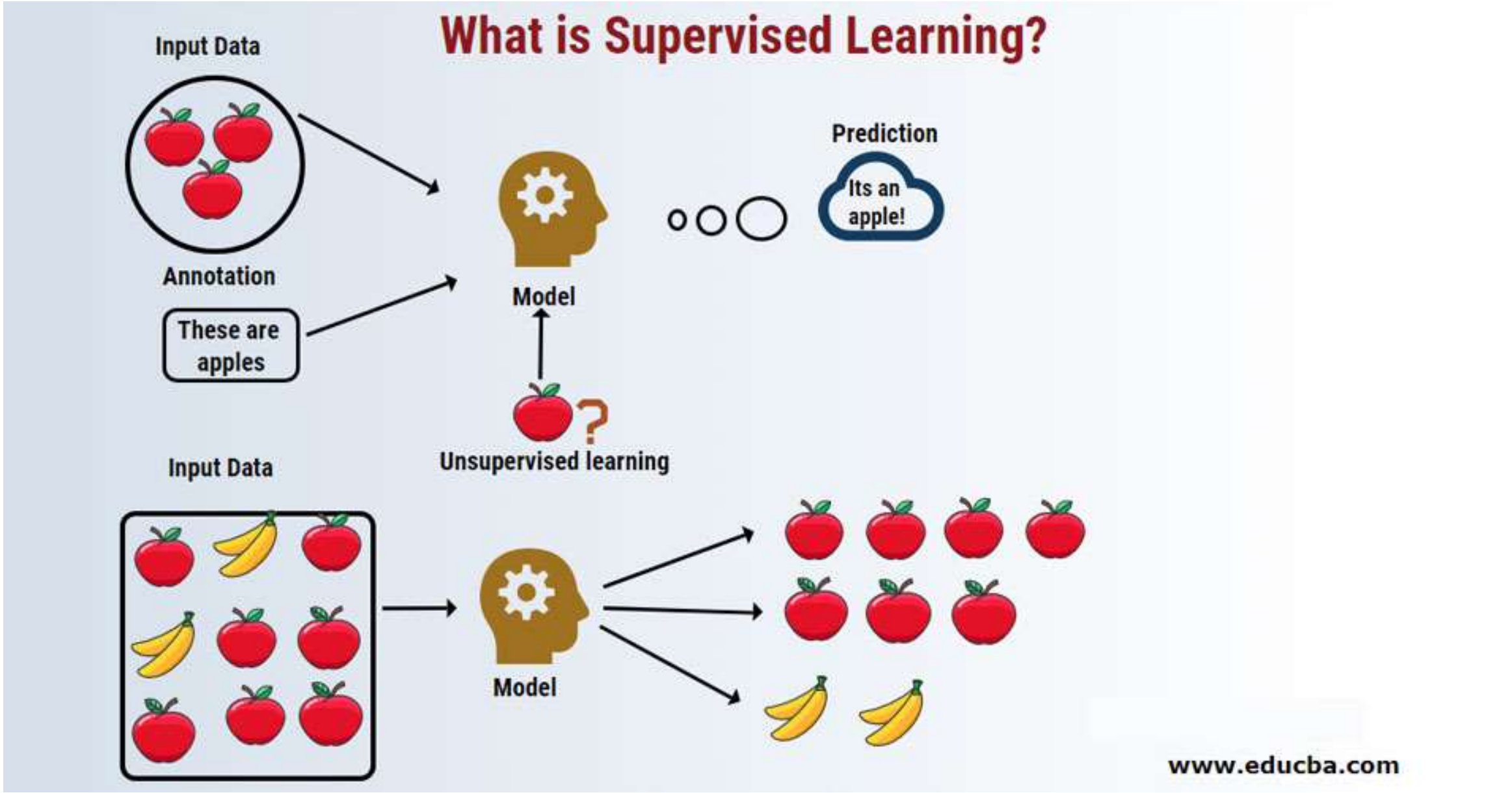
- **Module 2: Supervised Machine Learning Algorithms [09 Hours]**
- Introduction to Supervised Learning: Binary and Multi-Class Classification, Regression - Linear Regression and Nonlinear Regression, Logistic Regression, Naive Bayes Classification, Decision Trees (ID3, CART) and Random Forest, K-Nearest Neighborhood, Support Vector Machine (Linear and Non-linear Kernels).
- **Applications:** house price prediction systems, medical diagnosis systems, fault detection in machines, financial risk prediction.

Supervised Learning-Introduction

- **Supervised Learning** is the types of machine learning in which machines are trained with well “Labeled” training data and on the basis of that data, machine predict the output.
- In supervised learning, an algorithm is given a dataset containing input features (independent variables) and corresponding output labels (dependent variable).
- The model learns to associate inputs with outputs by minimizing the difference between its predictions and actual labels during training.
- Once trained, the model is evaluated using new data to assess its accuracy, generalization ability, and performance.
- Some real-world applications of supervised learning are **Risk Assessment, Fraud Detection, Spam filtering,** etc

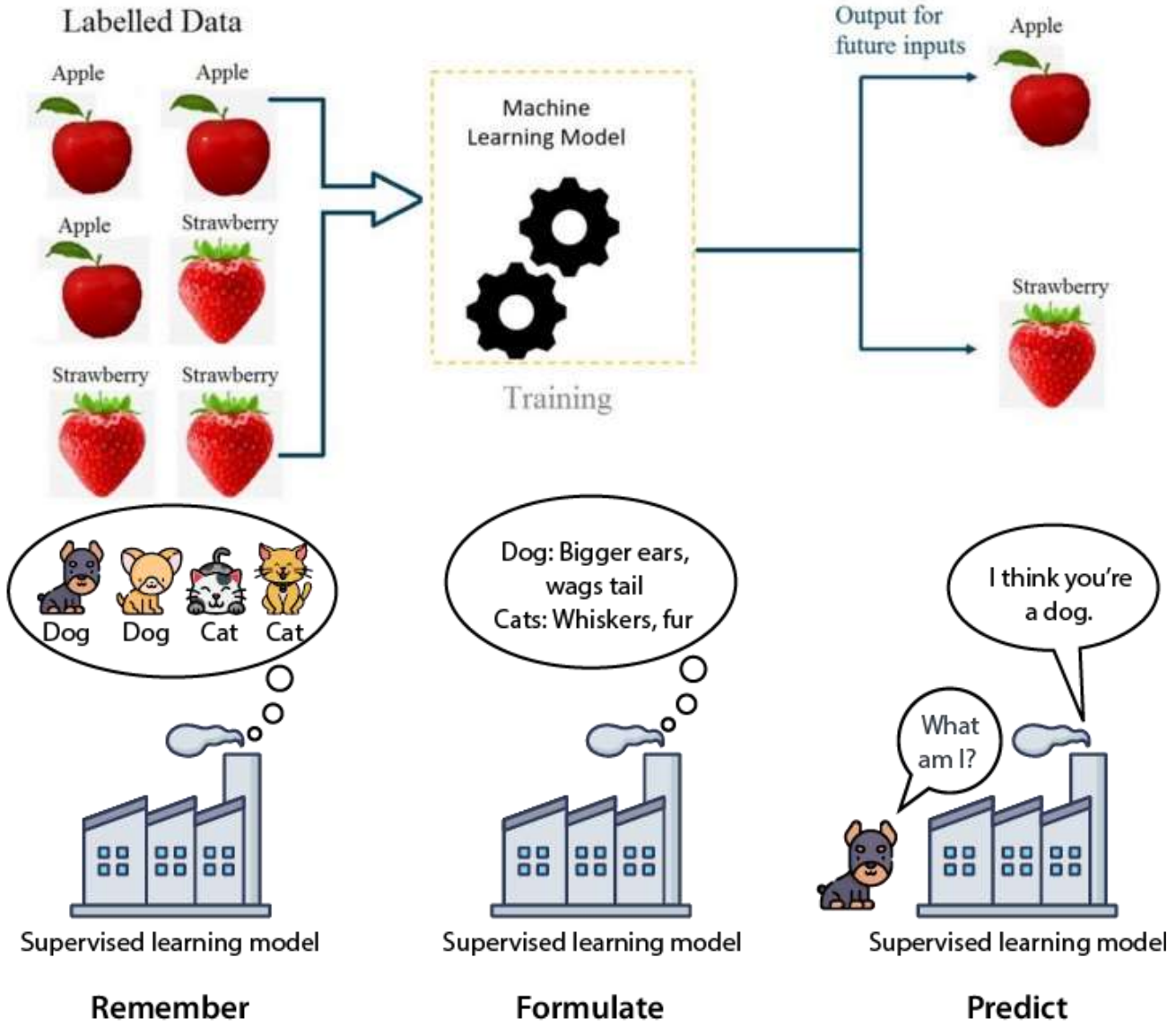
Supervised Learning-Introduction




What is Supervised Learning?
Input Data
Annotation
These are apples
Model
Prediction
Its an apple!
Unsupervised learning
Input Data
Model
www.educba.com

Supervised Learning Example

- previously collected data is needed in order to train our models.
- A model based on supervised learning would require both previous data and the previous results as input.
- By training with this data, the model helps in predicting results that are more accurate.



Supervised Learning-Introduction

- **Types of Supervised Learning Tasks**:
  - **Classification**: The task of predicting discrete labels or categories. Examples include spam detection, sentiment analysis, and handwriting recognition.
    - **Binary Classification**: Two possible labels (e.g., spam vs. not spam).
    - **Multiclass Classification**: More than two possible labels (e.g., classifying handwritten digits from 0 to 9).
  - **Regression**: The task of predicting continuous values. Examples include predicting house prices, stock prices, or temperature.

Examples for classification and regression

enjoy algorithms

Regression



What will be the temperature tomorrow?



Fahrenheit

Classification

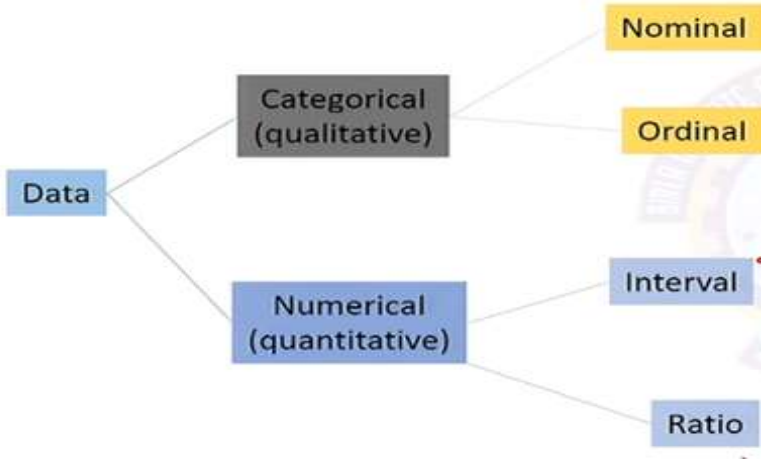


Will it be hot or cold tomorrow?



Fahrenheit

Variable Types



- **Categorical Variables**
  - These variables represent distinct categories or labels. The categories have no inherent order or ranking.
  - **Examples**: Gender (Male, Female), Colors (Red, Blue, Green), Types of vehicles (Car, Truck, Motorcycle).
  - **Subtypes**:
    - **Nominal Variables**: Categories have no logical order. Example: Eye color (Brown, Blue, Green).
    - **Ordinal Variables**: Categories have a meaningful order, but the differences between them aren’t measurable. Example: Satisfaction level (Low, Medium, High).

Variable Types

- **Numerical Variables**
  - These variables represent quantities and are expressed as numbers. They can be measured and ranked.
  - **Subtypes**:
    - **Discrete Variables**: These take on distinct or countable values. They are often whole numbers.
      - **Examples**: Number of children, Number of cars, Exam scores (if only integers are used).
    - **Continuous Variables**: These can take any value within a range and are often measured rather than counted.
      - **Examples**: Height, Weight, Temperature, Time.

Variable Types

- **Binary Variables**
  - A special case of categorical variables that have only two possible outcomes or categories.
  - **Examples**:
    - Yes/No questions.
    - Male/Female.
    - Success/Failure.

Variable Types

- **Interval Variables**
  - These variables are numerical, but the intervals between the values are meaningful and equal. However, interval variables do not have a true zero point, meaning that zero does not represent the complete absence of the quantity.
  - **Examples**:
    - Temperature in Celsius or Fahrenheit (0°C does not mean "no temperature").
    - Dates (Year 2000 and Year 2020).

Variable Types

- **Ratio Variables**
  - Like interval variables, ratio variables are numerical, but they have a meaningful zero point, allowing for both differences and ratios to be calculated.
  - **Examples**:
    - Weight (0 kg means no weight).
    - Height (0 meters means no height).
    - Income (0 dollars means no income).

Variable Types

- **Dummy Variables**
  - Categorical variables that are converted into numerical form (typically binary) for use in machine learning models. This process is called "dummy encoding" or "one-hot encoding."
  - **Example**:
    - If "Color" is a categorical variable with three categories (Red, Blue, Green), you can convert it into three dummy variables:
      - Red (1 if the color is red, 0 otherwise),
      - Blue (1 if the color is blue, 0 otherwise),
      - Green (1 if the color is green, 0 otherwise).

Terminology

- **Training Data**
  - The dataset used to train a model. It consists of input-output pairs, where the inputs are the features, and the outputs are the labels (or target values).
  - **Example**: In a housing price prediction model, the training data might include features like the size, number of bedrooms, and location, with the corresponding house price as the output.
- **Testing Data**
  - A separate dataset used to evaluate the model's performance after training. It contains input features but no labels are shown to the model during testing.
  - **Example**: After training, the model is tested on new house data to see how well it predicts prices.

Terminology

- **Features (Independent Variables, Predictors)**
  - The input variables used to make predictions. Features are the attributes or characteristics that the model uses to learn from.
  - **Example**: For predicting house prices, features might include square footage, number of rooms, and age of the house.
- **Labels (Dependent Variable, Target, Output)**
  - The output variable or the "correct" answer the model is trying to predict. In supervised learning, labels are provided in the training data.
  - **Example**: The house price in a house price prediction problem, or whether an email is spam or not in a spam classification task.

Terminology

- **Model**
  - The mathematical function or algorithm that is trained using the input data and makes predictions based on new input data.
  - **Example**: A linear regression model, decision tree, or neural network.
- **Hypothesis**
  - The function that maps input features to the predicted output based on the learned parameters. It's essentially the model's understanding of the relationship between input and output.
  - **Example**: In a linear regression model, the hypothesis could be a line that fits the data, such as $y=mx+b$.

Terminology

- **Loss Function (Cost Function)**
  - A function that measures how well the model's predictions match the actual labels. It calculates the difference between the predicted output and the actual output. The goal of training is to minimize this loss.
  - **Examples**:
    - **Mean Squared Error (MSE)**: Commonly used in regression tasks.
    - **Cross-Entropy Loss**: Used in classification tasks.
- **Optimization Algorithm**
  - The method used to minimize the loss function by adjusting the model's parameters. A popular optimization algorithm is **Gradient Descent**.
  - **Example**: Gradient descent iteratively updates model parameters to minimize the loss function.

Model Selection

- **Model selection** refers to the process of choosing the best machine learning algorithm or model for a given problem, based on its performance on the data.
- The **bias-variance tradeoff** plays a crucial role in model selection, as it helps balance underfitting and overfitting by understanding the sources of error in a model.

Model Selection

- **Smoothing** or **complexity parameters** are settings used to control the complexity of a model.
- These parameters help to balance the model's ability to fit the training data (complexity) with its ability to generalize to new, unseen data (smoothing).
- Proper tuning of these parameters is crucial for achieving a good model that avoids overfitting or underfitting.

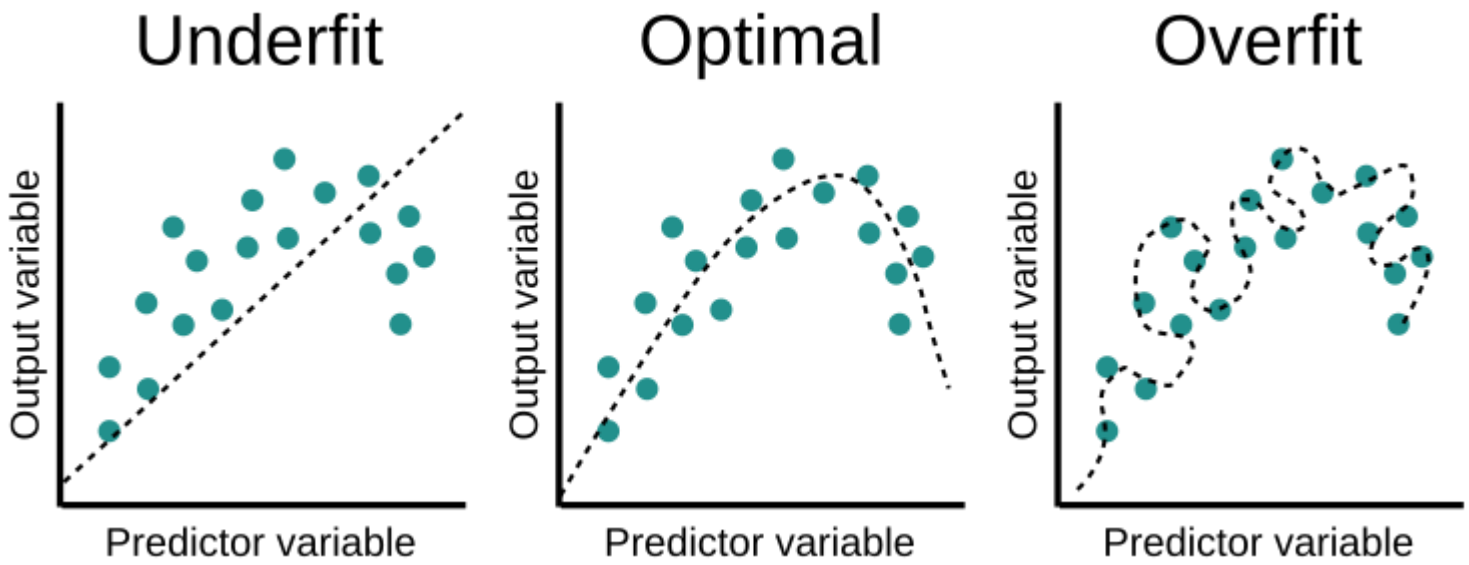
Model Selection

- **Overfitting**
  - When a model learns too much detail from the training data, including noise, and performs poorly on unseen data (test data). This occurs when the model is too complex and fits the training data too closely.
  - **Example**: A model might perfectly predict house prices in the training set but fails to generalize to new houses.

Model Selection

- **Underfitting**
  - When a model is too simple to capture the underlying patterns in the data, leading to poor performance on both the training and test data.
  - **Example**: A model that assumes house prices only depend on square footage and ignores all other important factors.

Model Selection



Model Selection

- **Smoothing** or **Complexity Parameters**
- **Multiplier of the Penalty Term (Regularization Parameter)**
  - Used in regularization techniques to penalize the complexity of the model. It controls the trade-off between fitting the training data well and keeping the model simpler to avoid overfitting.
- **Width of the Kernel (Kernel Width/Bandwidth)**
  - Controls the smoothness of the function estimated by a kernel-based method. It determines how much influence each training example has on the prediction.
- **Number of Basis Functions**
  - Controls the complexity of a model that uses basis functions to represent the input features. Basis functions transform the input space into a higher-dimensional space to better capture relationships.

Bias Variance Tradeoff

- **Bias**
  - Bias refers to the error due to overly simplistic assumptions in the learning algorithm. High bias means that the model is unable to capture the underlying patterns in the data, leading to **underfitting**.
  - Models with high bias will have low complexity, which may lead to missing important relationships in the data.
  - **Example**: A linear regression model trying to fit a highly nonlinear relationship will have high bias because it oversimplifies the data.

Bias Variance Tradeoff

- **Variance**
  - Variance refers to the model's sensitivity to small fluctuations in the training data. A model with high variance pays too much attention to the noise in the training data, leading to **overfitting**.
  - High-variance models tend to perform well on training data but poorly on unseen test data because they learn not only the underlying pattern but also the noise.
  - **Example**: A decision tree with too many branches might perfectly fit the training data, but when new data is introduced, it may fail to generalize well, showing high variance.

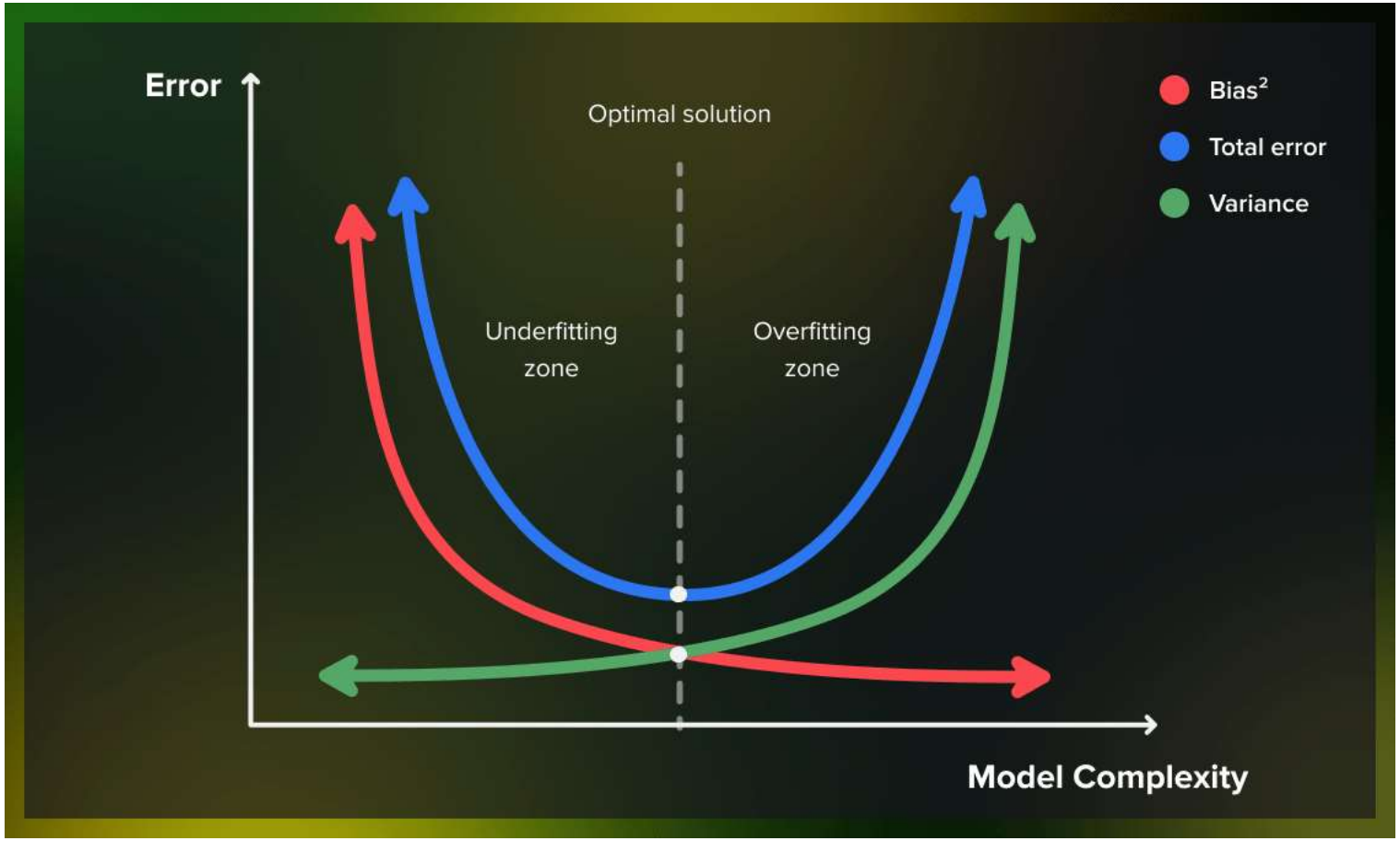
Bias Variance Tradeoff

- **Total Error (Prediction Error)**
  - The total error in a supervised learning model is a combination of bias, variance, and **irreducible error** (which is the noise inherent in the data that cannot be eliminated).
  - $\text{Error}=\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$

Bias Variance Tradeoff

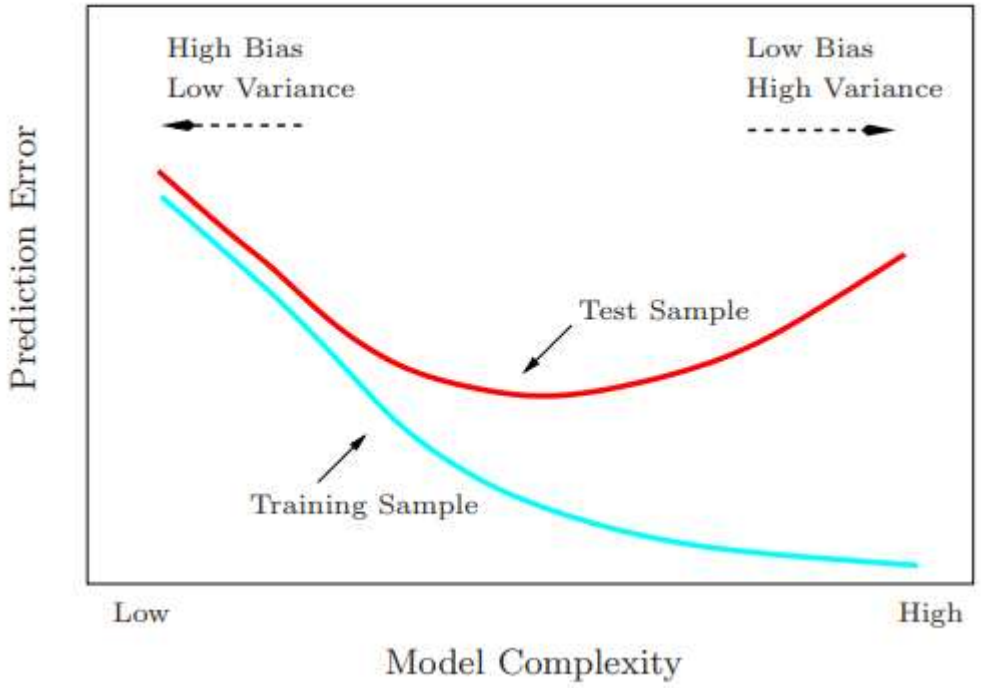
- If the model is **too simple** and has very **few parameters** then it may have **high bias** and **low variance**.
- On the other hand if the model has **large number of parameters** then it's going to have **high variance** and **low bias**.
- So, the right/good balance without overfitting and underfitting the data is needed.
- This tradeoff in complexity is why there is a tradeoff between bias and variance.
- An algorithm can't be more complex and less complex at the same time.

Bias Variance Tradeoff



Bias Variance Tradeoff

- Test and training error as a function of model complexity.



Linear Regression

- A linear regression model assumes that the regression function E(Y |X) is linear in the inputs X_1,... ,X_p.
- Linear Regression is a simple and powerful model for predicting a numeric response from a set of one or more independent variables.
- The linear model either assumes that the regression function E(Y |X) is linear, or that the linear model is a reasonable approximation.
- Fitting a linear regression model is all about finding the set of coefficients that best model *y* as a function of our features.

Linear Regression

- The linear regression model has the form

$$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j$$

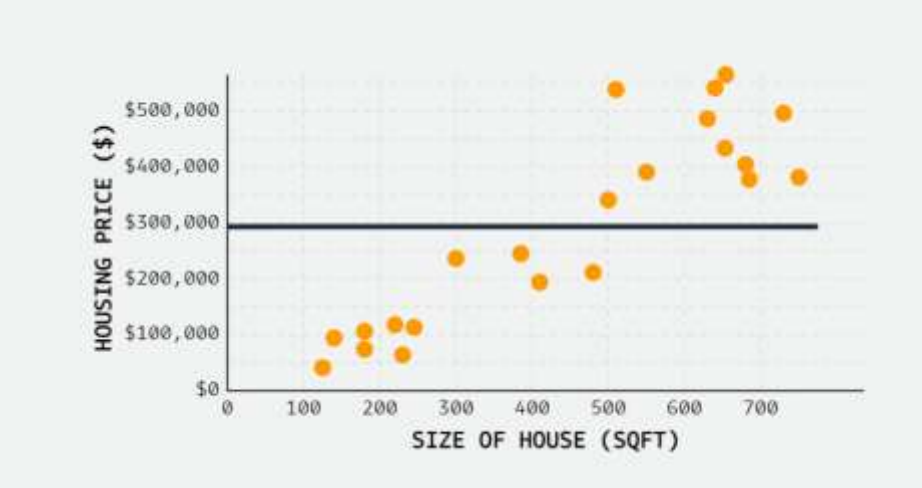
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p + \epsilon$$

- y: the dependent variable; the thing we are trying to predict.
- x_i: the independent variables: the features our model uses to model y.
- β_i: the coefficients ("weights") of our regression model. These are the foundations of our model. They are what our model "learns" during optimization.
- ϵ: the irreducible error in our model. A term that collects together all the unmodeled parts of our data.

Linear Regression

- Example: Predict housing price $\text{house-price} = \hat{\beta}_1 * sqft + \hat{\beta}_0$
- a very simple model, predicting the price of each house to be just the average house price in our dataset, ~$290,000, ignoring the different sizes of each house

$$\text{house-price} = 0 * sqft + 290000$$



Linear Regression

- These errors, or residuals, measure the distance between each observation and the predicted value for that observation.



Linear Regression

Typically we have a set of training data $(x_1, y_1) \ldots (x_N, y_N)$ from which to estimate the parameters β. Each $x_i = (x_{i1}, x_{i2}, \ldots, x_{ip})^T$ is a vector of feature measurements for the ith case. The most popular estimation method is *least squares*, in which we pick the coefficients $\beta = (\beta_0, \beta_1, \ldots, \beta_p)^T$ to minimize the residual sum of squares

$$\begin{aligned} \text{RSS}(\beta) &= \sum_{i=1}^{N} (y_i - f(x_i))^2 \\ &= \sum_{i=1}^{N} \Big(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \Big)^2. \end{aligned}$$

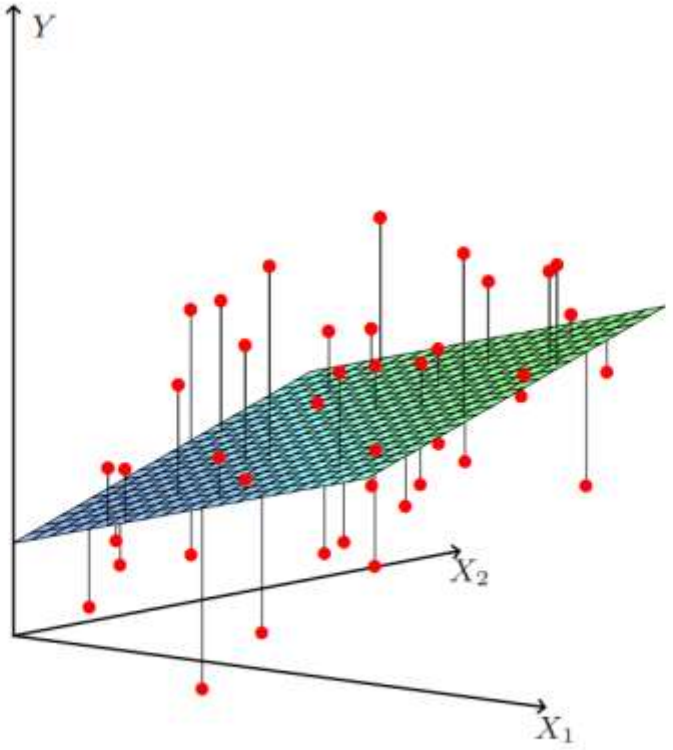
Linear Regression

- The goal of linear regression is reducing this error such that we find a line/surface that 'best' fits our data.



Linear Regression

- For two dimensional space



Linear least squares fitting with $X \in \mathbb{R}^2$. We seek the linear function of X that minimizes the sum of squared residuals from Y.

Linear Regression

- The model can be used to predict future values.

- x_i: 350

$$\hat{y} = 756.9 * 350 - 27153.8$$
$$\hat{y} = 237761$$



Linear Regression

- Example:

The slope is calculated using the following formula:

$$a = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Where:

- x_i = individual values of the input variable x
- y_i = individual values of the output variable y
- $\bar{x}$ = mean of the input values
- $\bar{y}$ = mean of the output values

This formula represents the covariance between x and y divided by the variance of x, which gives the slope of the regression line.

Linear Regression

- Example:

Once the slope a is calculated, the intercept is found using the formula:

$$b = \bar{y} - a\bar{x}$$

Where:

- $\bar{x}$ = mean of the input values x
- $\bar{y}$ = mean of the output values y
- a = slope calculated previously

Linear Regression

- Example:
- Dataset

Input - x	Output - y
1	3
2	5
3	7
4	9
5	11

Linear Regression

- Example:

Step 1: Calculate the mean of x and y:

$$\bar{x} = \frac{1+2+3+4+5}{5} = 3$$

$$\bar{y} = \frac{3+5+7+9+11}{5} = 7$$

Linear Regression

- Example:

Step 2: Calculate the slope (a):

We need to calculate both the numerator and denominator separately:

- **Numerator**:

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = (1-3)(3-7) + (2-3)(5-7) + (3-3)(7-7) + (4-3)(9-7) + (5-3)(11-7)$$

$$= (-2)(-4) + (-1)(-2) + (0)(0) + (1)(2) + (2)(4)$$

$$= 8 + 2 + 0 + 2 + 8 = 20$$

- **Denominator**:

$$\sum (x_i - \bar{x})^2 = (1-3)^2 + (2-3)^2 + (3-3)^2 + (4-3)^2 + (5-3)^2$$

$$= (-2)^2 + (-1)^2 + (0)^2 + (1)^2 + (2)^2$$

$$= 4 + 1 + 0 + 1 + 4 = 10$$

Now, the slope is:

$$a = \frac{20}{10} = 2$$

Linear Regression

- Example:

Step 3: Calculate the intercept (b):

$$b = \bar{y} - a\bar{x} = 7 - (2 \times 3) = 7 - 6 = 1$$

Final Result:

Using this formula, we get the same regression equation:

$$y = 2x + 1$$

Linear Regression

x	Actual y	Predicted $\hat{y}$	Residual $y - \hat{y}$	Residual²
1	3	$2(1) + 1 = 3$	$3 - 3 = 0$	$0^2 = 0$
2	5	$2(2) + 1 = 5$	$5 - 5 = 0$	$0^2 = 0$
3	7	$2(3) + 1 = 7$	$7 - 7 = 0$	$0^2 = 0$
4	9	$2(4) + 1 = 9$	$9 - 9 = 0$	$0^2 = 0$
5	11	$2(5) + 1 = 11$	$11 - 11 = 0$	$0^2 = 0$

$$RSS = \sum (y - \hat{y})^2$$

$$RSS = 0 + 0 + 0 + 0 + 0 = 0$$

$$MSE = \frac{RSS}{N}$$

$$MSE = \frac{0}{5} = 0$$

Linear Regression

Suppose we want to find the predicted y for a new x value of $x = 6$.

Step-by-Step Calculation:

Using the regression equation:

$$y = 2x + 1$$

Substitute $x = 6$ into the equation:

$$y = 2(6) + 1 = 12 + 1 = 13$$

Result:

For $x = 6$, the predicted y is $y = 13$.

This means that according to the regression model $y = 2x + 1$, when x is 6, the predicted value of y is 13.

Linear Regression

Simple Linear Regression (Univariate Linear Regression)

- **Univariate** means there is only one independent variable.
- The relationship between Y and X is modeled as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- Y is the dependent variable.
- X is the independent variable.
- β_0 is the intercept.
- β_1 is the slope (regression coefficient).
- ϵ is the error term (residual).

Linear Regression

Multivariate Linear Regression (Multiple Linear Regression)

- **Multivariate** means there are multiple independent variables.
- The model extends to:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

Where:

- $X_1, X_2, \ldots, X_p$ are the independent variables.
- $\beta_1, \beta_2, \ldots, \beta_p$ are the regression coefficients.

In both univariate and multivariate linear regression, the model assumes a **linear relationship** between the dependent and independent variables.

Linear Regression

- The linear model with p > 1 inputs is called the multiple linear regression model.

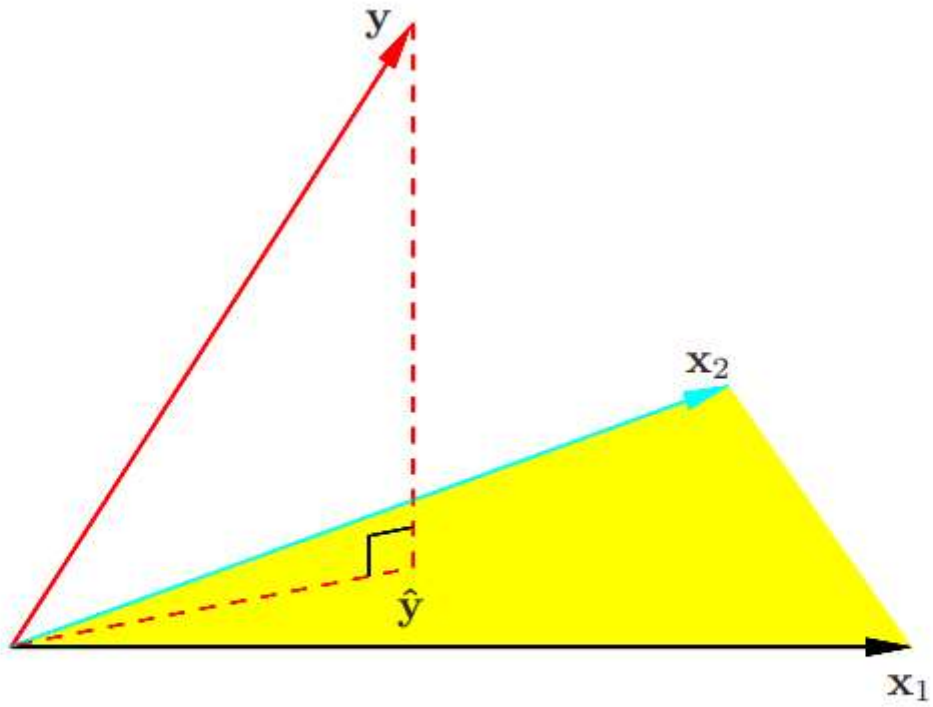
$$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j.$$

The least squares estimate and residuals are

$$\hat{\beta} = \frac{\sum_1^N x_i y_i}{\sum_1^N x_i^2},$$

$$r_i = y_i - x_i \hat{\beta}.$$

Linear Regression



The N-dimensional geometry of least squares regression with two predictors. The outcome vector $\mathbf{y}$ *is orthogonally projected onto the hyperplane spanned by the input vectors* $\mathbf{x}_1$ *and* $\mathbf{x}_2$*. The projection* $\hat{\mathbf{y}}$ *represents the vector of the least squares predictions*

Non-Linear Regression

- Non-linear regression is used when the relationship between the dependent and independent variables is **not linear**. The regression function can take many different forms (exponential, polynomial, logarithmic, etc.).

Non-Linear Regression

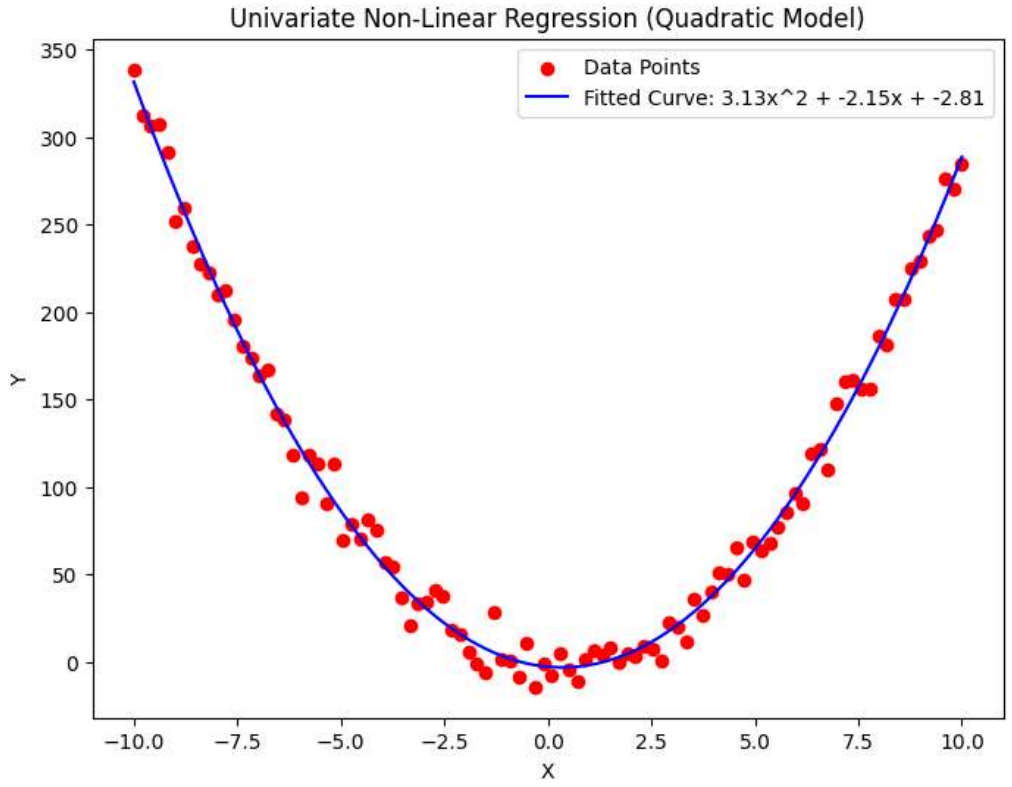
Univariate Non-Linear Regression

- There is only one independent variable, but the relationship is non-linear.
- For example, the relationship might be quadratic:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

- The model fits a **curve** rather than a straight line.

Non-Linear Regression



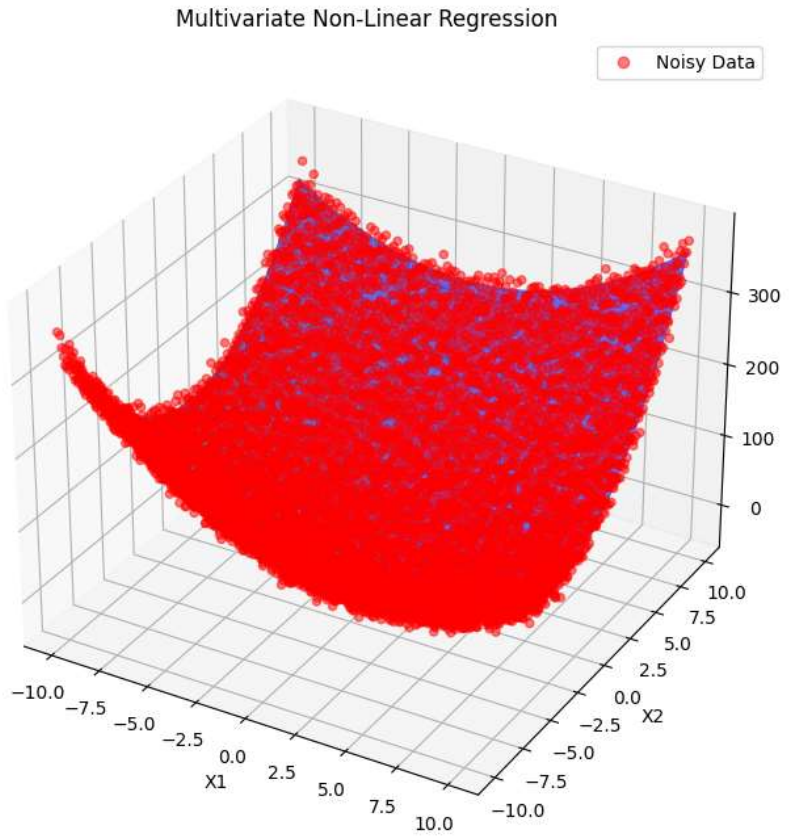
Non-Linear Regression

Multivariate Non-Linear Regression

- There are multiple independent variables, and their relationship with Y is non-linear.
- For example, a model could include exponential terms, interaction terms, or logarithmic transformations:

$$Y = \beta_0 + \beta_1 e^{X_1} + \beta_2 X_2^2 + \epsilon$$

Non-Linear Regression



a * x1**2 + b * x2**2 + c * x1 * x2 + d * x1 + e * x2 + f

Linear Regression

Cost function-

- The different values for weights or coefficient of lines (a_0, a_1) gives the different line of regression, and the cost function is used to estimate the values of the coefficient for the best fit line.
- Cost function optimizes the regression coefficients or weights. It measures how a linear regression model is performing.
- We can use the cost function to find the accuracy of the **mapping function**, which maps the input variable to the output variable. This mapping function is also known as **Hypothesis function**.

For Linear Regression, we use the **Mean Squared Error (MSE)** cost function, which is the average of squared error occurred between the predicted values and actual values. It can be written as:

$$MSE = 1\frac{1}{N}\sum_{i=1}^{n}(y_i - (a_1x_i + a_0))^2$$

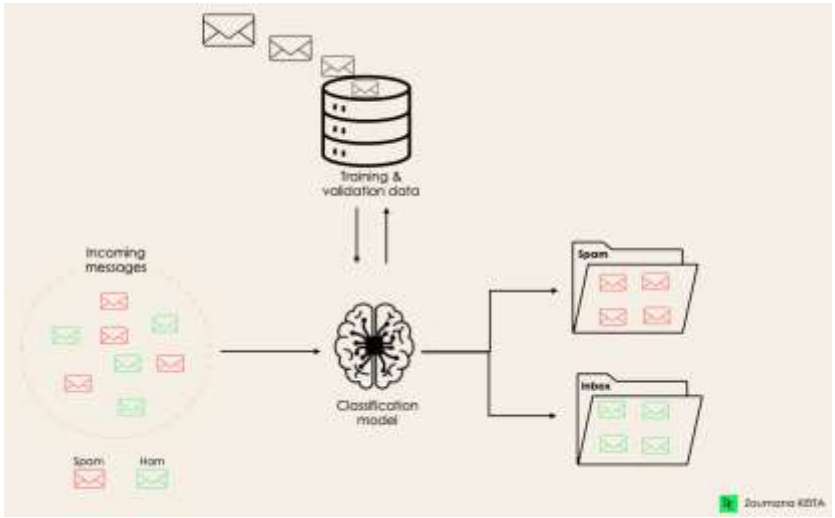
N=Total number of observation

Yi = Actual value

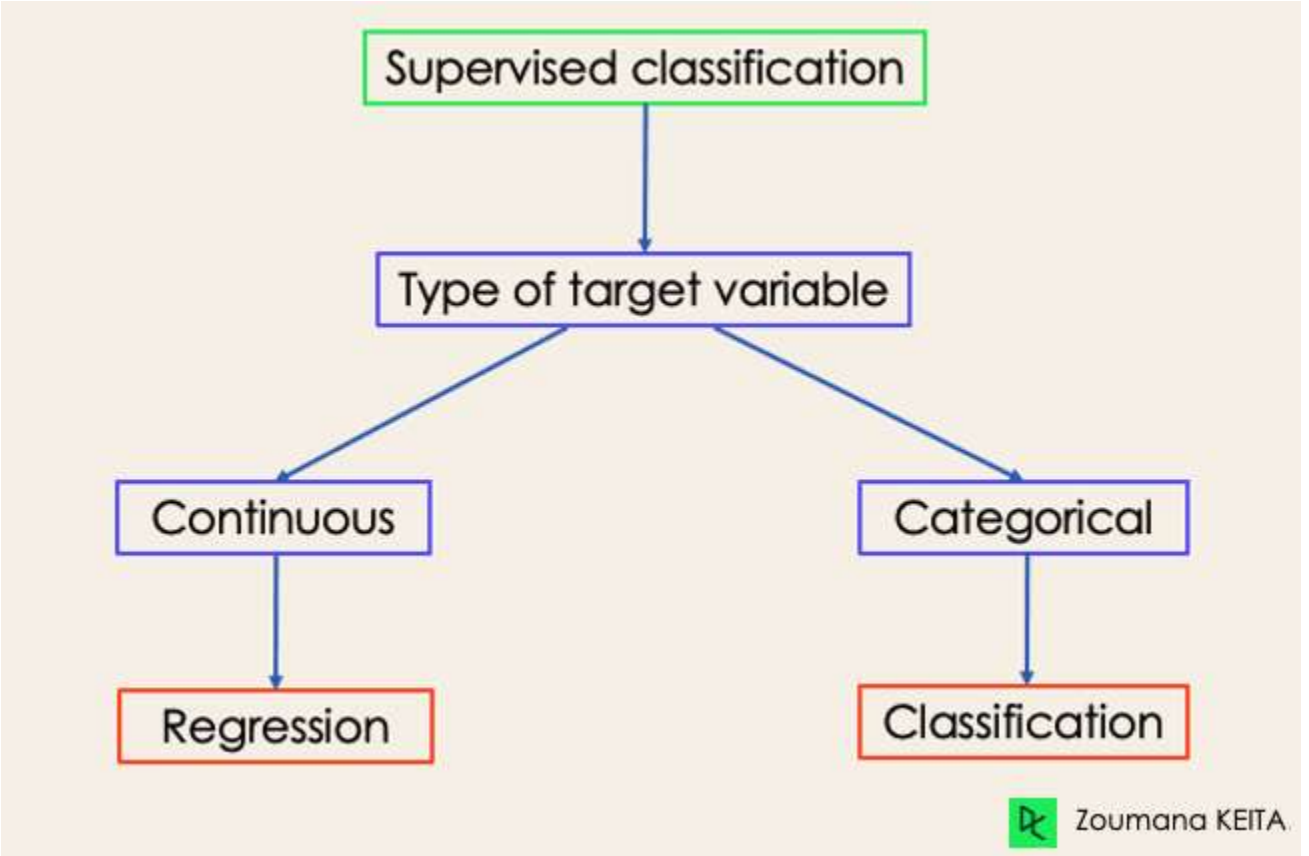
$(a1x_i+a_0)$= Predicted value.

Classification in Machine Learning

- Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data. In classification, the model is fully trained using the training data, and then it is evaluated on test data before being used to perform prediction on new unseen data.
- For instance, an algorithm can learn to predict whether a given email is spam or ham (no spam), as illustrated below.



Classification Vs. Regression



Machine Learning Classification in Real Life

Healthcare

Training a machine learning model on historical patient data can help healthcare specialists accurately analyze their diagnoses:

- During the COVID-19 pandemic, machine learning models were implemented to efficiently predict whether a person had COVID-19 or not.

Education

- Education is one of the domains dealing with the most textual, video, and audio data. This unstructured information can be analyzed with the help of Natural Language technologies to perform different tasks such as:
- The classification of documents per category.

Sustainable agriculture

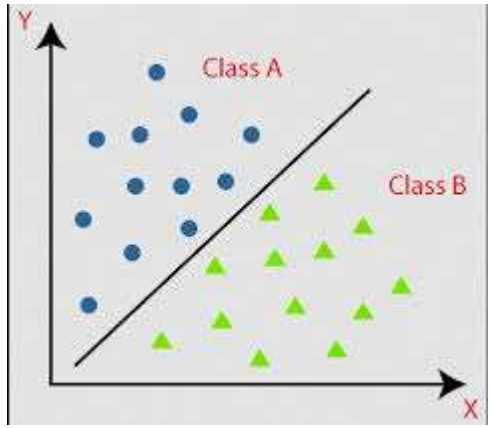
- Agriculture is one of the most valuable pillars of human survival. Introducing sustainability can help improve farmers' productivity at a different level without damaging the environment:
- By using classification models to predict which type of land is suitable for a given type of seed.

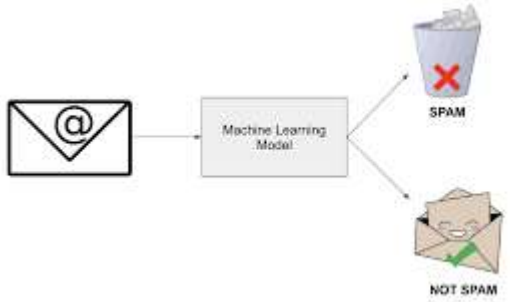
Different Types of Classification

Binary Classification

The goal is to classify the input data into two mutually exclusive categories. The training data in such a situation is labeled in a binary format: true and false; positive and negative; O and 1; spam and not spam, etc. depending on the problem being tackled.

For instance, we might want to detect whether a given image is a truck or a boat.

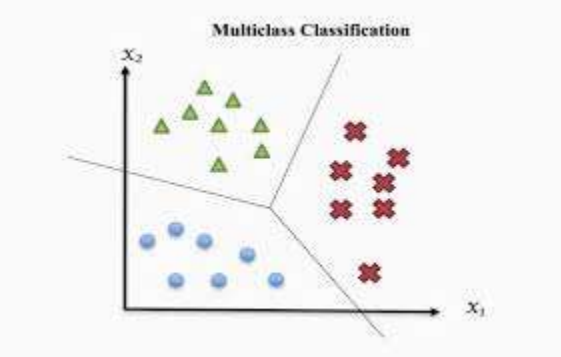


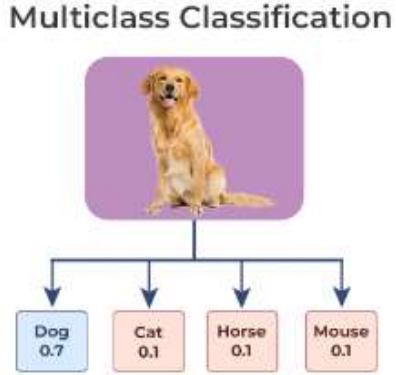


Logistic Regression and Support Vector Machines algorithms are natively designed for binary classifications. However, other algorithms such as K-Nearest Neighbors and Decision Trees can also be used for binary classification.

Multi-Class Classification

The multi-class classification, on the other hand, has at least two mutually exclusive class labels, where the goal is to predict to which class a given input example belongs to. In the following case, the model correctly classified the image to be a plane.

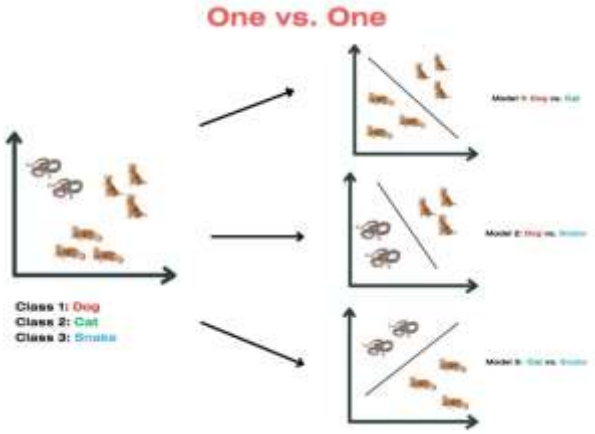




- Most of the binary classification algorithms can be also used for multi-class classification. These algorithms include but are not limited to:
  - Random Forest
  - Naive Bayes
  - K-Nearest Neighbors
  - Gradient Boosting
  - SVM
  - Logistic Regression.

Multi-Class Classification

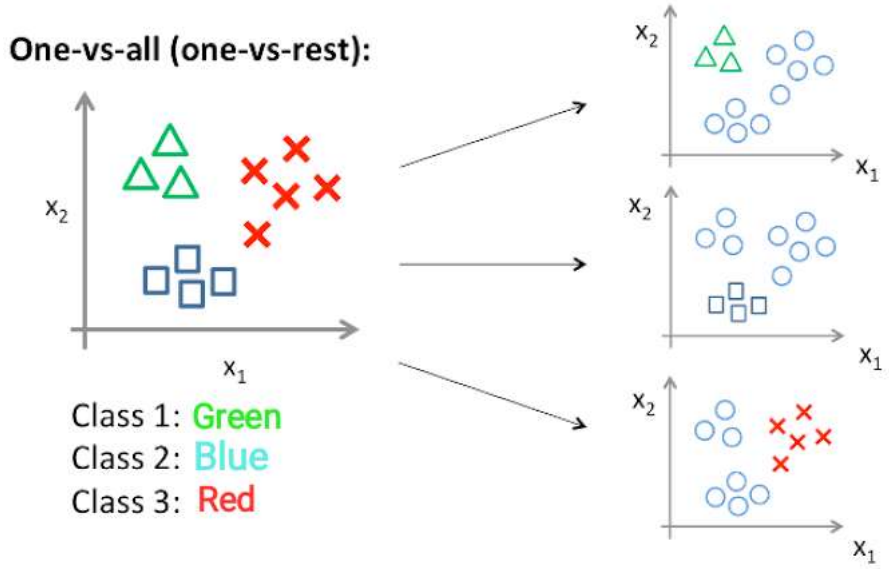
- *SVM and Logistic Regression do not support multi-class classification by default.*
- *We can apply binary transformation approaches such as one-versus-one and one-versus-all to adapt native binary classification algorithms for multi-class classification tasks.*
- **One-versus-one**: this strategy trains as many classifiers as there are pairs of labels. If we have a 3-class classification, we will have three pairs of labels, thus three classifiers, as shown below.



- For N labels, we will have Nx(N-1)/2 classifiers. Each classifier is trained on a single binary dataset, and the final class is predicted by a majority vote between all the classifiers. One-vs-one approach works best for SVM and other kernel-based algorithms.

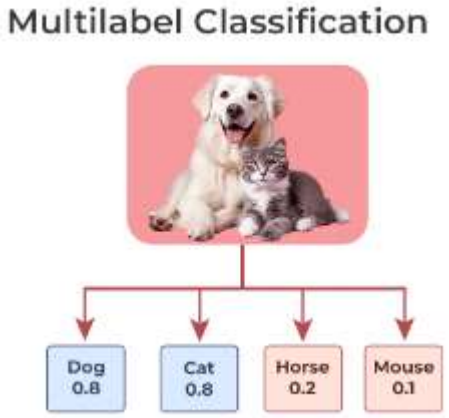
Multi-Class Classification

- **One-versus-rest**: at this stage, we start by considering each label as an independent label and consider the rest combined as only one label. With 3-classes, we will have three classifiers.
- In general, for N labels, we will have **N** binary classifiers.



Multi-Label Classification

- In multi-label classification tasks, we try to predict 0 or more classes for each input example. In this case, there is no mutual exclusion because the input example can have more than one label.
- Such a scenario can be observed in different domains, such as auto-tagging in Natural Language Processing, where a given text can contain multiple topics. Similarly to computer vision, an image can contain multiple objects



Multi-Label Classification

- It is not possible to use multi-class or binary classification models to perform multi-label classification. However, most algorithms used for those standard classification tasks have their specialized versions for multi-label classification. We can cite:
- *Multi-label Decision Trees*
- *Multi-label Gradient Boosting*
- *Multi-label Random Forests*

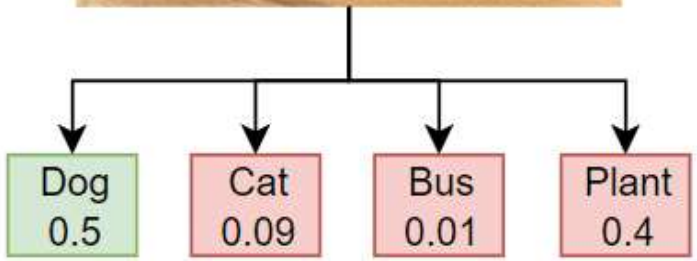
Binary Classification





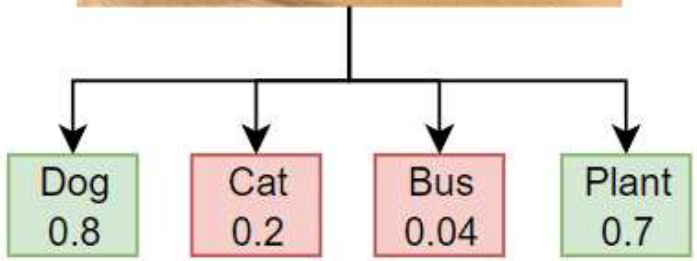
Multiclass Classification





Multilabel Classification





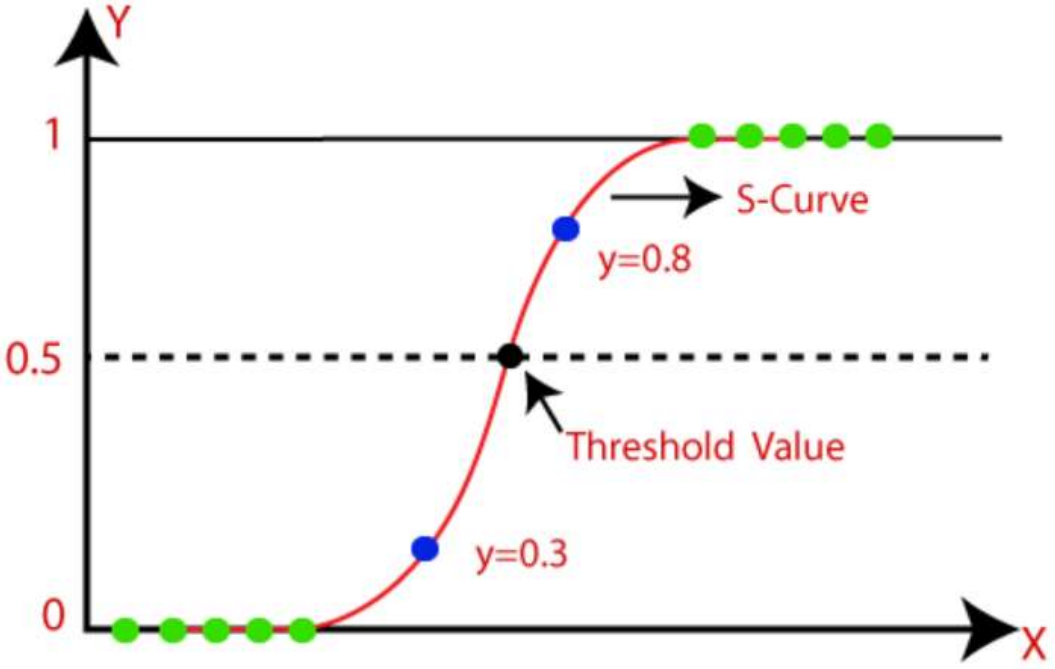
Logistic Regression

- Supervised Learning technique.
- It is used for predicting the categorical dependent variable using a given set of independent variables.
- Predicts the output of a categorical dependent variable.
- It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1**.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.

Logistic Regression

- Logistic Regression is much similar to the Linear Regression except that how they are used.
- Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems**.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification.

Logistic Regression



Note: Logistic regression uses the concept of predictive modeling as regression; therefore, it is called logistic regression, but is used to classify samples; Therefore, it falls under the classification algorithm.

Logistic Regression

Logistic Function (Sigmoid Function):

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.

Assumptions for Logistic Regression:

- The dependent variable must be categorical in nature.
- The independent variable should not have multi-collinearity.

Logistic Regression

- Example
- Dataset

Study Hours (x)	Passed (y)
2	0
4	0
6	1
8	1
10	1

Logistic Regression

- Example

1. **Model Setup**:
   Logistic regression models the probability of passing the exam using the sigmoid function:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

   Where:

   - β_0 is the intercept (bias term),
   - β_1 is the coefficient for the feature (Study Hours).

2. **Parameter Estimation**:
   To estimate β_0 and β_1, we would typically use maximum likelihood estimation (MLE) or a machine learning library to fit the model. For simplicity, let's assume we have already estimated the parameters:

$$\beta_0 = -4$$

$$\beta_1 = 0.8$$

Logistic Regression

- Example

1. For $X = 2$:

$$z = \beta_0 + \beta_1 \cdot 2 = -4 + 0.8 \cdot 2 = -2.4$$

$$P(Y = 1|X = 2) = \frac{1}{1 + e^{-(-2.4)}} = \frac{1}{1 + e^{2.4}} = \frac{1}{1 + 11.02} = 0.083$$

2. For $X = 4$:

$$z = \beta_0 + \beta_1 \cdot 4 = -4 + 0.8 \cdot 4 = -0.8$$

$$P(Y = 1|X = 4) = \frac{1}{1 + e^{-(-0.8)}} = \frac{1}{1 + e^{0.8}} = \frac{1}{1 + 2.23} = 0.310$$

3. For $X = 6$:

$$z = \beta_0 + \beta_1 \cdot 6 = -4 + 0.8 \cdot 6 = 0.8$$

$$P(Y = 1|X = 6) = \frac{1}{1 + e^{-(0.8)}} = \frac{1}{1 + e^{0.8}} = \frac{1}{1 + 0.45} = 0.690$$

Logistic Regression

- Example

4. For $X = 8$:

$$z = \beta_0 + \beta_1 \cdot 8 = -4 + 0.8 \cdot 8 = 2.4$$

$$P(Y = 1|X = 8) = \frac{1}{1 + e^{-(2.4)}} = \frac{1}{1 + 0.09} = 0.916$$

5. For $X = 10$:

$$z = \beta_0 + \beta_1 \cdot 10 = -4 + 0.8 \cdot 10 = 4$$

$$P(Y = 1|X = 10) = \frac{1}{1 + e^{-(4)}} = \frac{1}{1 + 0.018} = 0.982$$

Step 3: Make Predictions

To classify the results, we set a decision threshold of 0.5:

- If $P(Y = 1|X) > 0.5$, predict $Y = 1$ (Pass),
- Otherwise, predict $Y = 0$ (Fail).

Naïve Bayes Classifier

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object**.
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles**.

Naïve Bayes Classifier

Why is it called Naïve Bayes?

The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:

- **Naïve**: It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes**: It is called Bayes because it depends on the principle of Bayes' Theorem.

Naïve Bayes Classifier

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.

$$P(A|B)=\frac{P(B|A)P(A)}{P(B)}$$

Where,

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.

Naïve Bayes Classifier

Working of Naïve Bayes' Classifier:

1. Convert the given dataset into frequency tables.
2. Generate Likelihood table by finding the probabilities of given features.
3. Now, use Bayes theorem to calculate the posterior probability.

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	4

Frequency table for the Weather Conditions:

Naïve Bayes Classifier

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	4

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Naïve Bayes Classifier

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

P(Yes|Sunny)= P(Sunny|Yes)*P(Yes)/P(Sunny)

P(Sunny|Yes)= 3/10= 0.3

P(Sunny)= 0.35

P(Yes)=0.71

Applying Bayes'theorem:

P(Yes|Sunny) = 0.3*0.71/0.35= **0.60**

Naïve Bayes Classifier

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

P(No|Sunny)= P(Sunny|No)*P(No)/P(Sunny)

P(Sunny|NO)= 2/4=0.5

P(No)= 0.29

P(Sunny)= 0.35

Applying Bayes'theorem: P(No|Sunny)= 0.5*0.29/0.35 = **0.41**

So as we can see from the above calculation that **P(Yes|Sunny)>P(No|Sunny)**

Hence on a Sunny day, Player can play the game.

Naïve Bayes Classifier

Example No.	Color	Type	Origin	Stolen?
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	No
3	Red	Sports	Domestic	Yes
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	No
7	Yellow	SUV	Imported	Yes
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	Sports	Imported	Yes

Naïve Bayes Classifier

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

$$X = (x_1, x_2, x_3, ..., x_n)$$

$$P(y|x_1, ..., x_n) = \frac{P(x_1|y)P(x_2|y)...P(x_n|y)P(y)}{P(x_1)P(x_2)...P(x_n)}$$

$$P(y|x_1, ..., x_n) \propto P(y) \prod_{i=1}^{n} P(x_i|y)$$

$$y = argmax_y P(y) \prod_{i=1}^{n} P(x_i|y)$$

Naïve Bayes Classifier

Frequency Table

		Stolen?	
		Yes	No
Color	Red	3	2
	Yellow	2	3



Likelihood Table

		Stolen?	
		P(Yes)	P(No)
Color	Red	3/5	2/5
	Yellow	2/5	3/5

Naïve Bayes Classifier

Frequency Table

		Stolen?	
		Yes	No
Type	Sports	4	2
	SUV	1	3



Likelihood Table

		Stolen?	
		P(Yes)	P(No)
Type	Sports	4/5	2/5
	SUV	1/5	3/5

Naïve Bayes Classifier

Frequency Table

		Stolen?	
		Yes	No
Origin	Domestic	2	3
	Imported	3	2

Likelihood Table

		Stolen?	
		P(Yes)	P(No)
Origin	Domestic	2/5	3/5
	Imported	3/5	2/5

Naïve Bayes Classifier

Color	Type	Origin	Stolen
Red	SUV	Domestic	?

Probability of Stolen:

$$P(\text{Stolen}|\text{Red, SUV, Domestic}) \propto P(\text{Red}|\text{Stolen}) \cdot P(\text{SUV}|\text{Stolen}) \cdot P(\text{Domestic}|\text{Stolen}) \cdot P(\text{Stolen})$$

$$= \frac{3}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} \cdot 0.5 = 0.024$$

Probability of Not Stolen:

$$P(\text{Not Stolen}|\text{Red, SUV, Domestic}) \propto P(\text{Red}|\text{Not Stolen}) \cdot P(\text{SUV}|\text{Not Stolen}) \cdot P(\text{Domestic}|\text{Not Stolen}) \cdot P(\text{Not Stolen})$$

$$= \frac{2}{5} \cdot \frac{3}{5} \cdot \frac{3}{5} \cdot 0.5 = 0.036$$

Naïve Bayes Classifier

Now, we normalize these values to get the actual probabilities.

$$P(\text{Stolen}|\text{Red, SUV, Domestic}) = \frac{0.024}{0.024 + 0.036} = 0.4$$

$$P(\text{Not Stolen}|\text{Red, SUV, Domestic}) = \frac{0.036}{0.024 + 0.036} = 0.6$$

Conclusion

The probability of a "Red SUV Domestic" vehicle being **stolen** is **0.4**, while the probability of it **not being stolen** is **0.6**.

Naïve Bayes Classifier

- Find the probability of a **Plastic, Round, Red, Locally Made** item being defective

Example No.	Material	Shape	Color	Made Locally?	Defective?
1	Plastic	Round	Red	Yes	Yes
2	Plastic	Round	Red	Yes	No
3	Plastic	Square	Blue	No	Yes
4	Metal	Round	Blue	No	No
5	Plastic	Square	Blue	Yes	Yes
6	Metal	Round	Red	Yes	No
7	Metal	Square	Red	No	Yes
8	Plastic	Round	Blue	Yes	No
9	Metal	Square	Blue	No	No
10	Plastic	Round	Red	No	Yes

Naïve Bayes Classifier

Advantages of Naïve Bayes Classifier:

- Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.
- It can be used for Binary as well as Multi-class Classifications.
- It performs well in Multi-class predictions as compared to the other Algorithms.
- It is the most popular choice for **text classification problems**.

Disadvantages of Naïve Bayes Classifier:

- Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

Naïve Bayes Classifier

Applications of Naïve Bayes Classifier:

- It is used for **Credit Scoring**.
- It is used in **medical data classification**.
- It can be used in **real-time predictions** because Naïve Bayes Classifier is an eager learner.
- It is used in Text classification such as **Spam filtering** and **Sentiment analysis**.

Naïve Bayes Classifier – Multi class classification

In multi-class classification, we have multiple possible classes $C_1, C_2, \ldots, C_k$ for each data point, and we want to predict the probability of each class given some feature vector $X = (x_1, x_2, \ldots, x_n)$.

Using Bayes' theorem, the probability of each class C_i given X can be calculated as:

$$P(C_i|X) = \frac{P(X|C_i) \cdot P(C_i)}{P(X)}$$

where:

- $P(C_i|X)$ is the posterior probability of class C_i given the features X.
- $P(X|C_i)$ is the likelihood of observing the features X if the data belongs to class C_i.
- $P(C_i)$ is the prior probability of class C_i, which can be estimated from the data as the relative frequency of that class.
- $P(X)$ is the probability of observing the features X across all classes, and serves as a normalization factor.

In practice, we often ignore $P(X)$ when doing classification because it's the same for all classes and doesn't affect the relative comparison of class probabilities.

Naïve Bayes Classifier – Multi class classification

Calculating Class Probabilities in Multi-Class Naive Bayes

For a new data point with feature vector $X = (x_1, x_2, \ldots, x_n)$, we calculate the posterior probability for each class C_i. The class with the highest posterior probability is chosen as the predicted class. In multi-class Naive Bayes, we assume independence among the features given the class (Naive Bayes assumption). Thus, the likelihood $P(X|C_i)$ becomes:

$$P(X|C_i) = \prod_{j=1}^{n} P(x_j|C_i)$$

Then, the posterior for each class C_i can be calculated as:

$$P(C_i|X) \propto P(C_i) \cdot \prod_{j=1}^{n} P(x_j|C_i)$$

Naïve Bayes Classifier – Multi class classification

Compute Prior Probabilities $P(C_i)$

- Calculate the probability of each class C_i based on the training data.
- $P(C_i) = \frac{\text{Number of examples in class } C_i}{\text{Total number of examples}}$

Calculate Likelihoods $P(x_j|C_i)$

- For each feature x_j, compute the likelihood for each class. This depends on the type of feature:
  - **For categorical features**: Use frequency counts to estimate $P(x_j|C_i)$.
  - **For continuous features**: Use a probability distribution (like Gaussian) to estimate $P(x_j|C_i)$ by calculating the mean and variance of x_j for each class.

Naïve Bayes Classifier – Multi class classification

Combine Priors and Likelihoods

- Multiply the prior $P(C_i)$ by the product of likelihoods $\prod_{j=1}^{n} P(x_j|C_i)$ to get the unnormalized posterior for each class:

$$P(C_i|X) \propto P(C_i) \cdot \prod_{j=1}^{n} P(x_j|C_i)$$

Normalize (Optional, for Probability Values)

- To obtain the actual probability values (summing to 1), divide each $P(C_i|X)$ by the sum across all classes:

$$P(C_i|X) = \frac{P(C_i) \cdot \prod_{j=1}^{n} P(x_j|C_i)}{\sum_{k=1}^{K} P(C_k) \cdot \prod_{j=1}^{n} P(x_j|C_k)}$$

Naïve Bayes Classifier – Multi class classification

Suppose we have a dataset with three classes: C_1, C_2, and C_3, and two features x_1 and x_2.

For a new instance with features $X = (x_1 = \text{Low}, x_2 = \text{High})$, we calculate:

1. **Prior Probabilities** (assuming frequencies from the dataset):
   - $P(C_1) = 0.3$
   - $P(C_2) = 0.5$
   - $P(C_3) = 0.2$

Naïve Bayes Classifier – Multi class classification

2. **Likelihoods**:
   - For C_1:
     - $P(x_1 = \text{Low}|C_1) = 0.6$
     - $P(x_2 = \text{High}|C_1) = 0.3$
   - For C_2:
     - $P(x_1 = \text{Low}|C_2) = 0.4$
     - $P(x_2 = \text{High}|C_2) = 0.5$
   - For C_3:
     - $P(x_1 = \text{Low}|C_3) = 0.7$
     - $P(x_2 = \text{High}|C_3) = 0.2$

Naïve Bayes Classifier – Multi class classification

3. **Calculate Posterior Probabilities:**

- For C_1:

$$P(C_1|X) \propto P(C_1) \cdot P(x_1 = \text{Low}|C_1) \cdot P(x_2 = \text{High}|C_1) = 0.3 \cdot 0.6 \cdot 0.3 = 0.054$$

- For C_2:

$$P(C_2|X) \propto P(C_2) \cdot P(x_1 = \text{Low}|C_2) \cdot P(x_2 = \text{High}|C_2) = 0.5 \cdot 0.4 \cdot 0.5 = 0.1$$

- For C_3:

$$P(C_3|X) \propto P(C_3) \cdot P(x_1 = \text{Low}|C_3) \cdot P(x_2 = \text{High}|C_3) = 0.2 \cdot 0.7 \cdot 0.2 = 0.028$$

Naïve Bayes Classifier – Multi class classification

4. **Normalization (Optional)**:
   - The sum of unnormalized posteriors is $0.054 + 0.1 + 0.028 = 0.182$.
   - Normalized probabilities:
     - $P(C_1|X) = \frac{0.054}{0.182} \approx 0.297$
     - $P(C_2|X) = \frac{0.1}{0.182} \approx 0.549$
     - $P(C_3|X) = \frac{0.028}{0.182} \approx 0.154$

The classifier would predict the class with the highest posterior probability, which is C_2 with a probability of approximately 0.549.

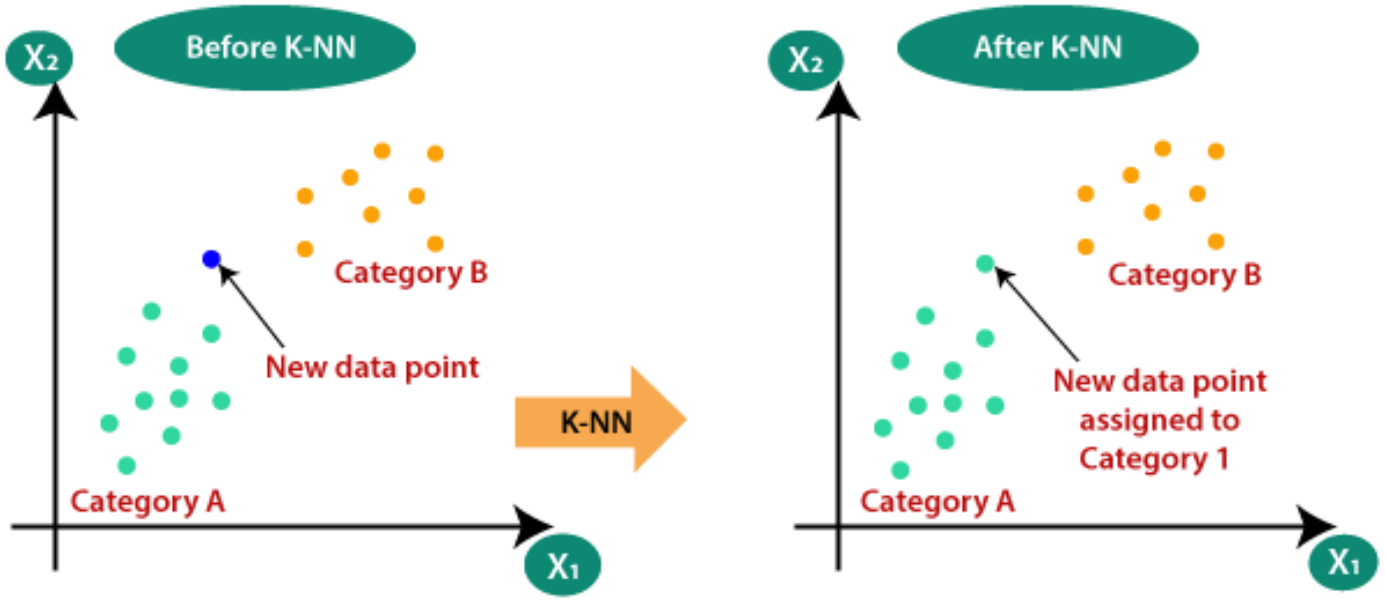
K-Nearest Neighbour(KNN)

- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



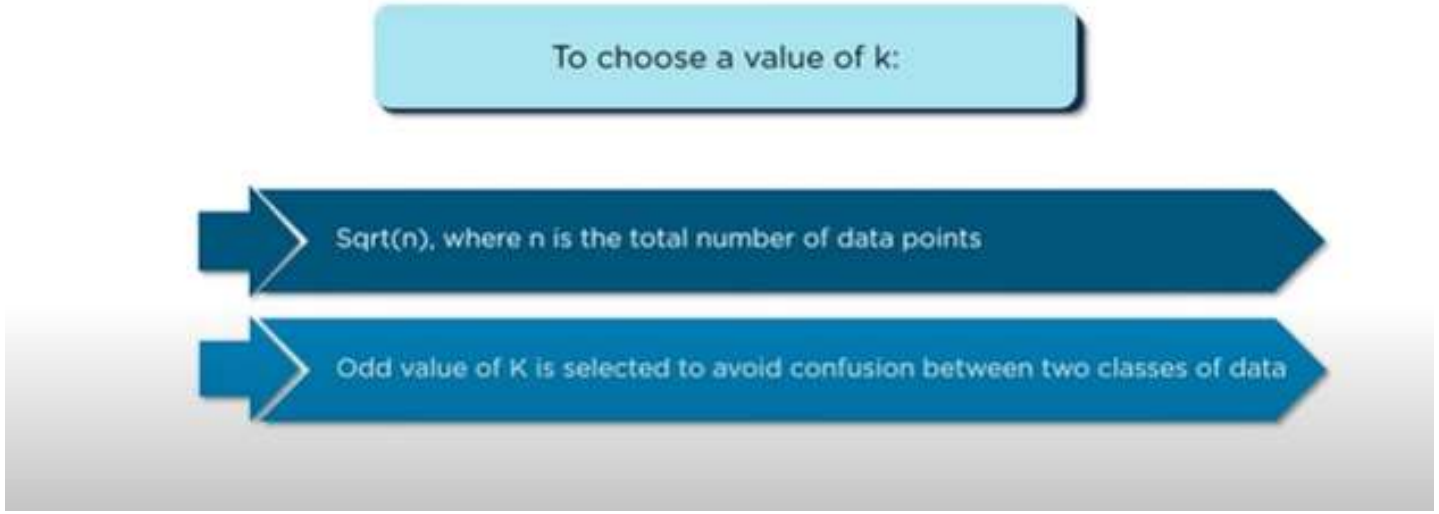
Why do we need a K-NN Algorithm?



How does K-NN work?

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

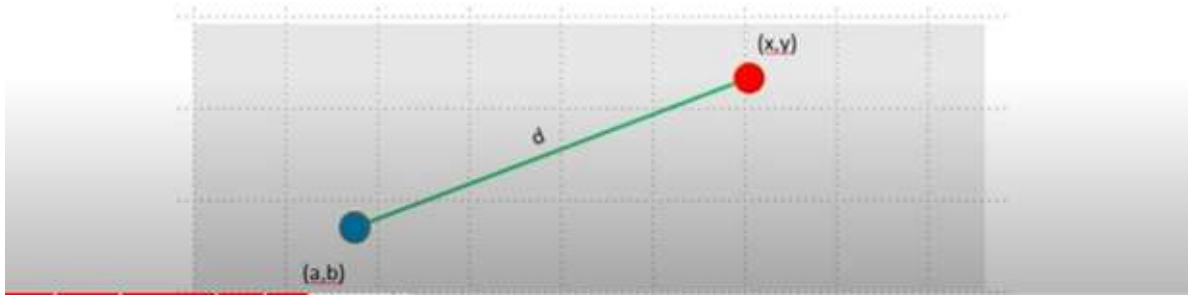
How to Choose k value?



Euclidean Distance Formula

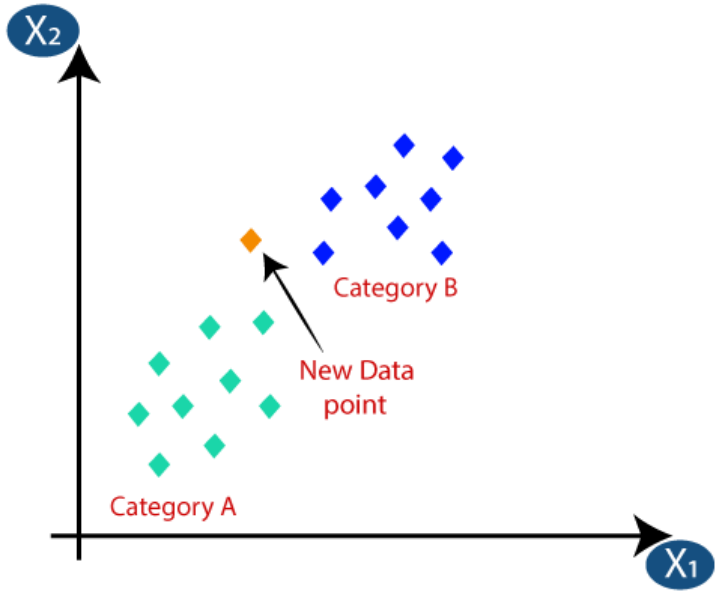
According to the **Euclidean distance** formula, the **distance** between two points in the plane with coordinates (x, y) and (a, b) is given by:

$$dist(d) = \sqrt{(x - a)^2 + (y - b)^2}$$



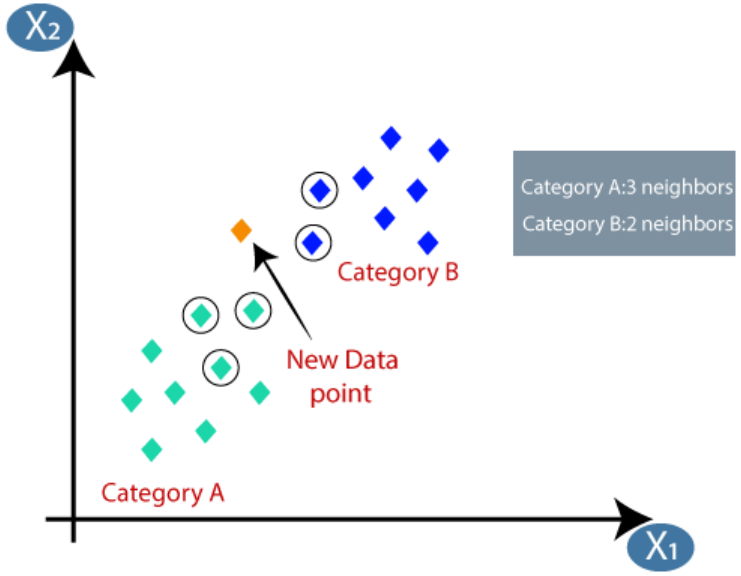
- Suppose we have a new data point and we need to put it in the required category.

Consider the below image:



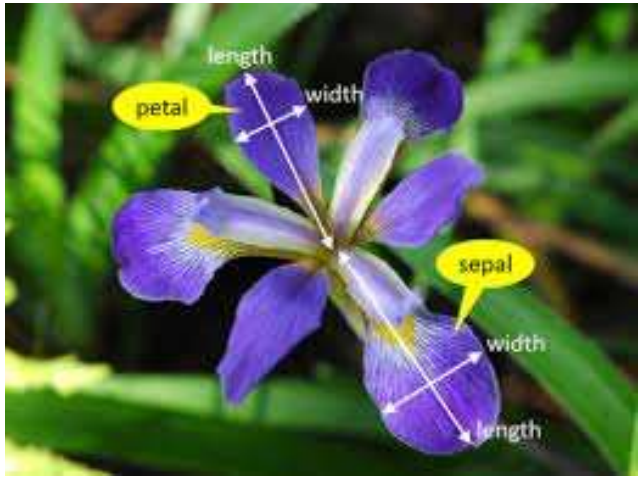
- Firstly, we will choose the number of neighbors, so we will choose the k=5.
- Next, we will calculate the **Euclidean distance** between the data points.

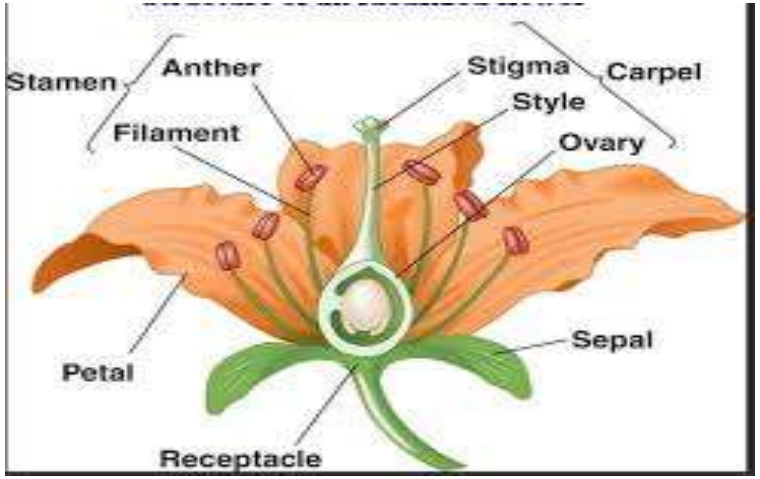
- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B.
- Consider the below image:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor







Sepal Length	Sepal Width	Species
5.2	3.1	?

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

Sepal Length	Sepal Width	Species
5.2	3.1	?

Step 1: Find Distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608
5.1	3.8	Setosa	0.707
7.2	3.0	Virginica	2.002
5.4	3.4	Setosa	0.36
5.1	3.3	Setosa	0.22
5.4	3.9	Setosa	0.82
7.4	2.8	Virginica	2.22
6.1	2.8	Verscicolor	0.94
7.3	2.9	Virginica	2.1
6.0	2.7	Verscicolor	0.89
5.8	2.8	Virginica	0.67
6.3	2.3	Verscicolor	1.36
5.1	2.5	Verscicolor	0.60
6.3	2.5	Verscicolor	1.25
5.5	2.4	Verscicolor	0.75

x1	y1	x2	y2	Distance
5.3	3.7	5.2	3.1	0.608276
5.1	3.8	5.2	3.1	0.707107
7.2	3	5.2	3.1	2.002498
5.4	3.4	5.2	3.1	0.360555
5.1	3.3	5.2	3.1	0.223607
5.4	3.9	5.2	3.1	0.824621
7.4	2.8	5.2	3.1	2.22036
6.1	2.8	5.2	3.1	0.948683
7.3	2.9	5.2	3.1	2.109502
6	2.7	5.2	3.1	0.894427
5.8	2.8	5.2	3.1	0.67082
6.3	2.3	5.2	3.1	1.360147
5.1	2.5	5.2	3.1	0.608276
6.3	2.5	5.2	3.1	1.252996
5.5	2.4	5.2	3.1	0.761577

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 2: Find Rank

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 3: Find the Nearest Neighbor

If k = 1 – Setosa

If k = 2 – Setosa

If k = 5 – Setosa

How to select the value of K in the K-NN Algorithm?

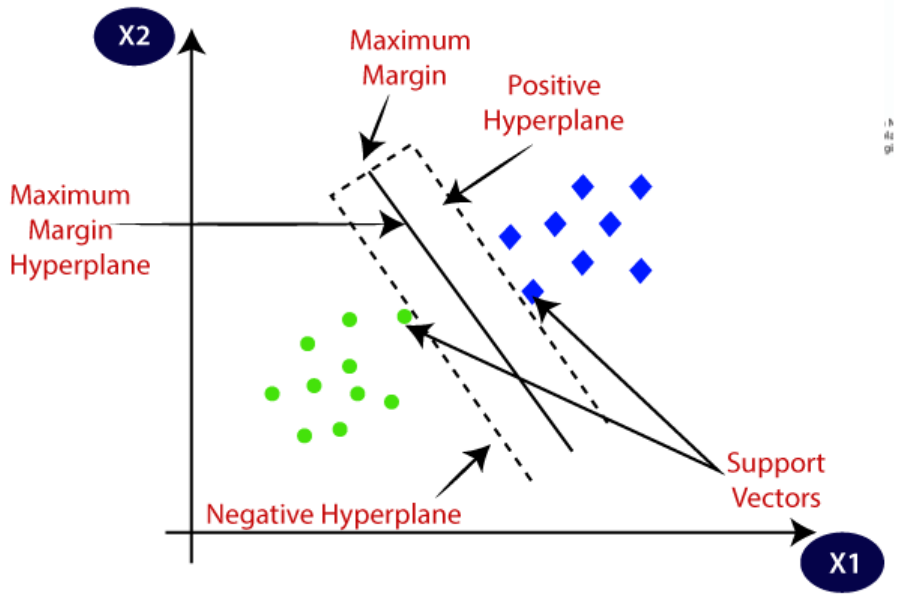
- Below are some points to remember while selecting the value of K in the K-NN algorithm:
  - There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
  - A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
  - Large values for K are good, but it may find some difficulties.

Support Vector Machine

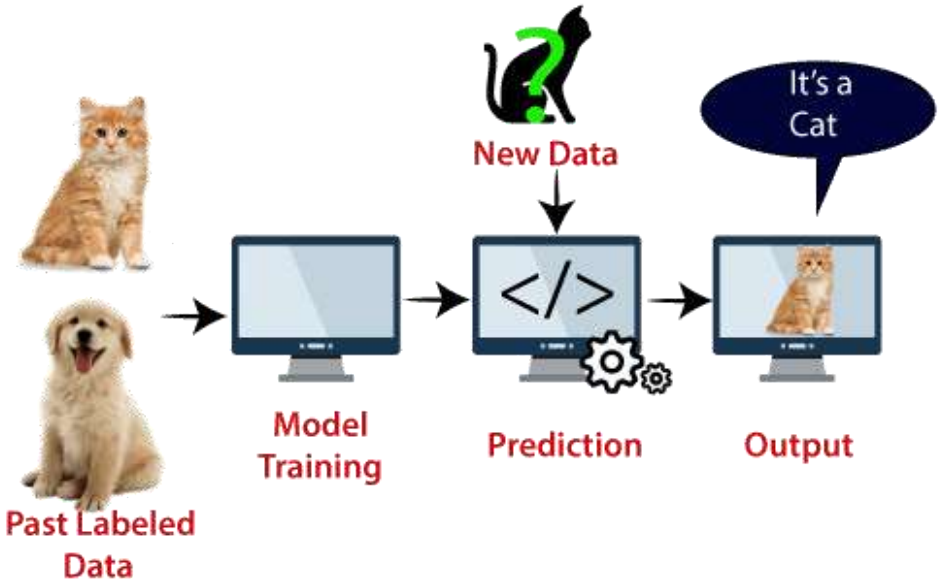
- Support Vector Machine or SVM is one of the most popular **Supervised Learning algorithms**, which is used for Classification as well as Regression problems. However, primarily, it is used for **Classification problems** in Machine Learning.
- The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. **This best decision boundary is called a hyperplane.**

SVM

- SVM chooses the extreme points/vectors that help in creating the hyperplane.
- These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine.
- Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane.

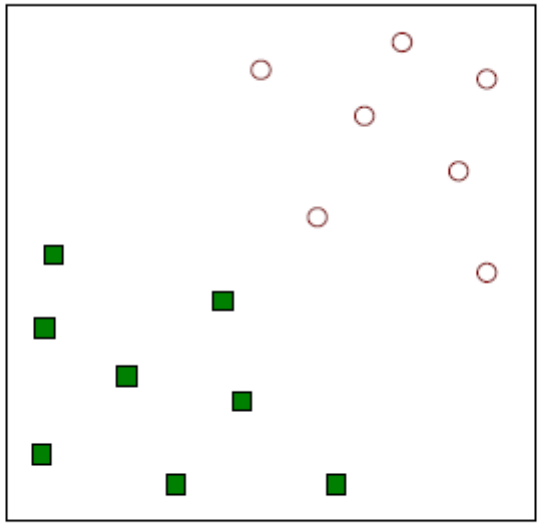

X2
Maximum Margin
Positive Hyperplane
Maximum Margin Hyperplane
Support Vectors
Negative Hyperplane
X1

Example



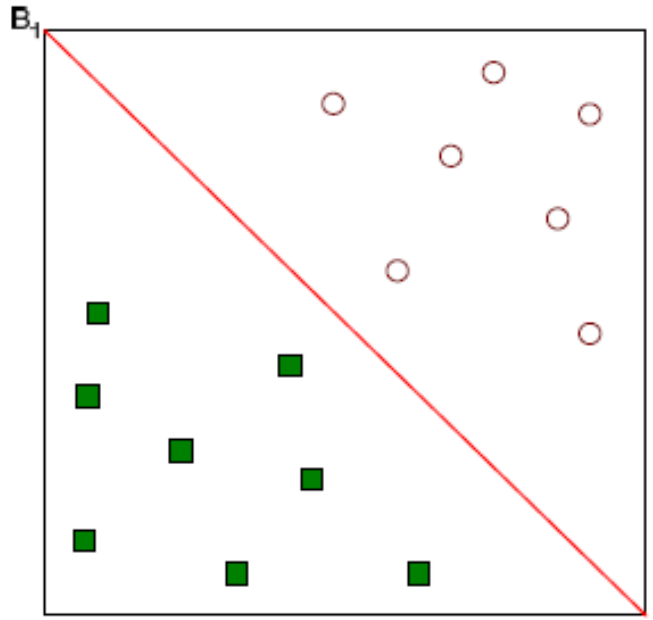
SVM algorithm can be used for **Face detection, image classification, text categorization,** etc.

Support Vector Machine



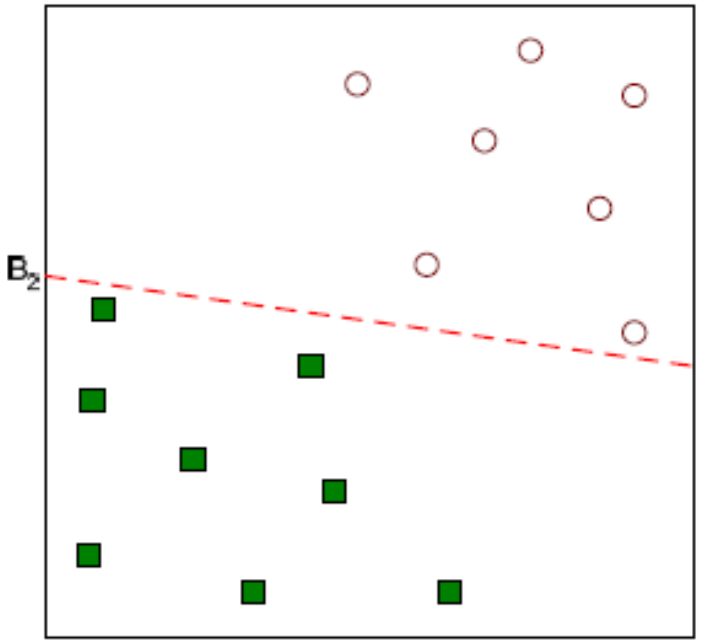
- Find a linear hyperplane (decision boundary) that will separate the data

Support Vector Machine



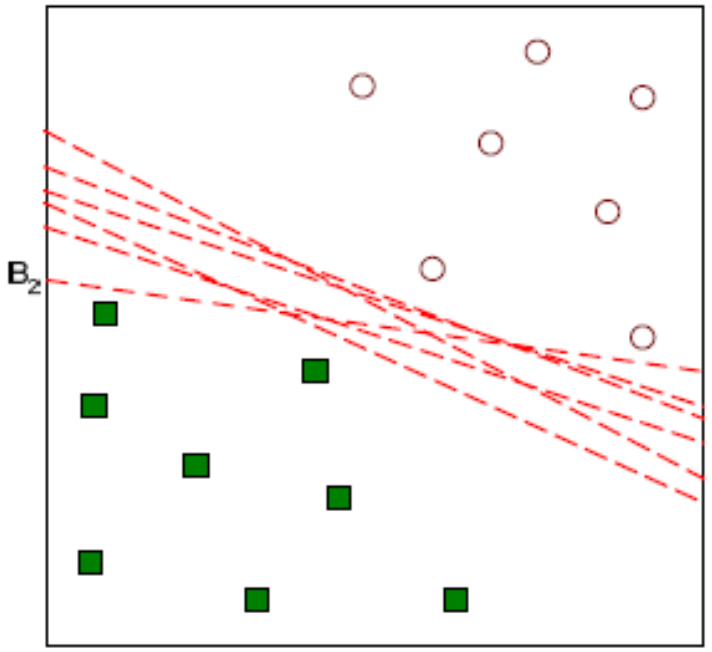
- One Possible Solution

Support Vector Machine



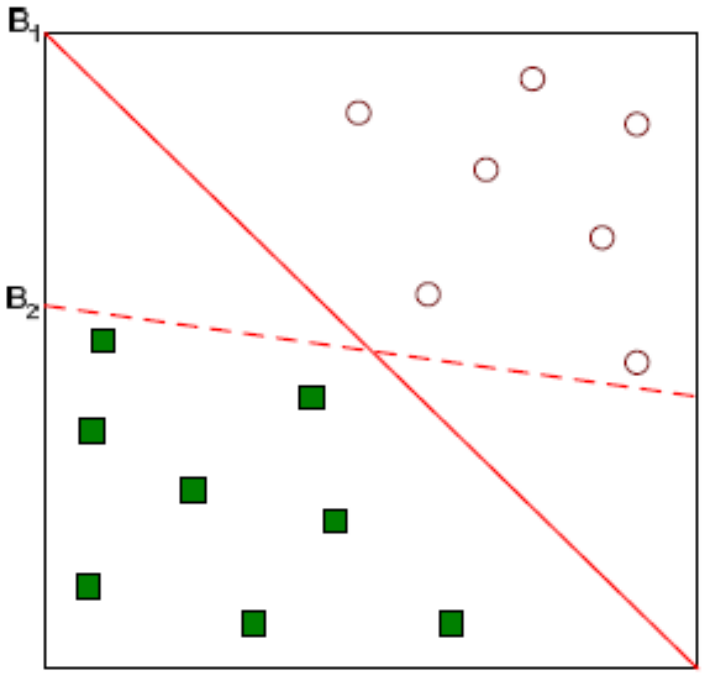
- Another possible solution

Support Vector Machine



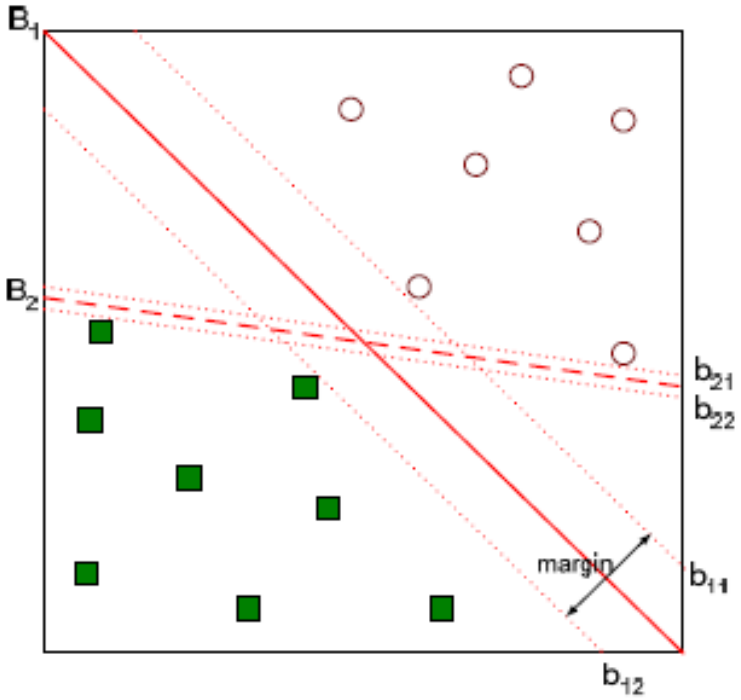
- Other possible solutions

Support Vector Machine



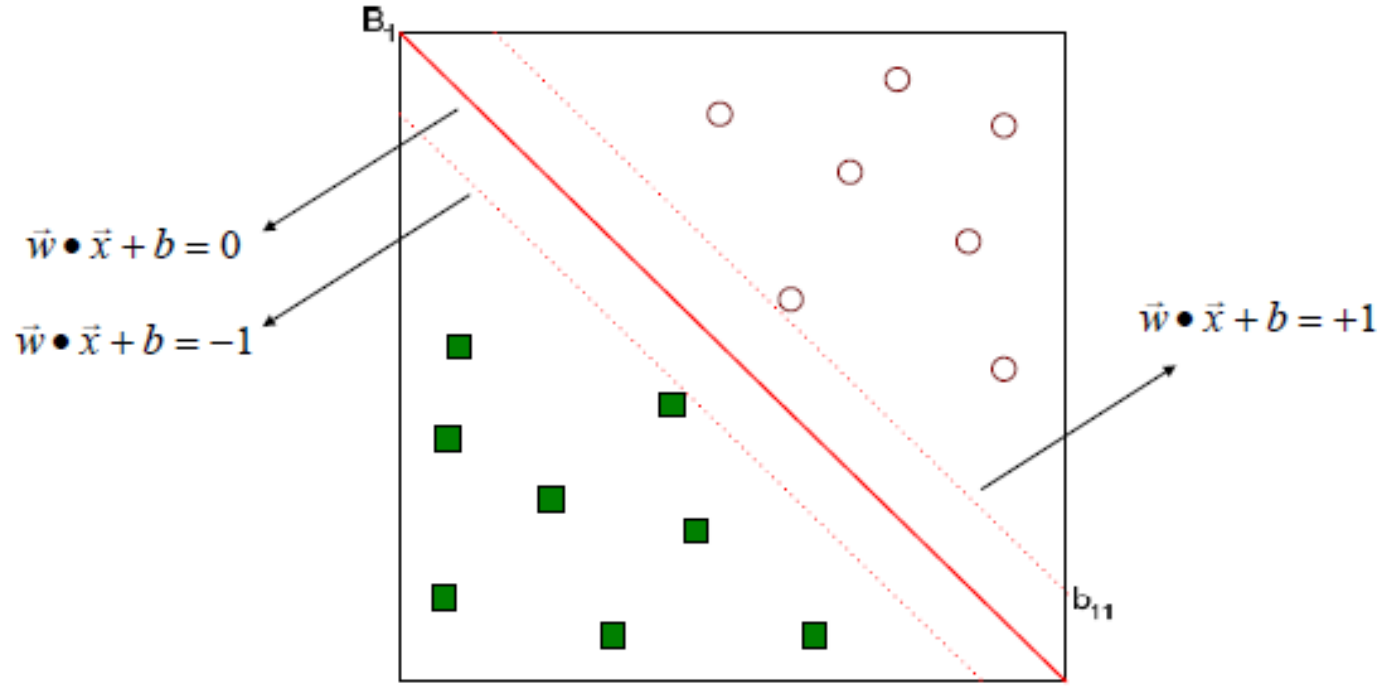
- Which one is better? B1 or B2?
- How do you define better?

Support Vector Machine



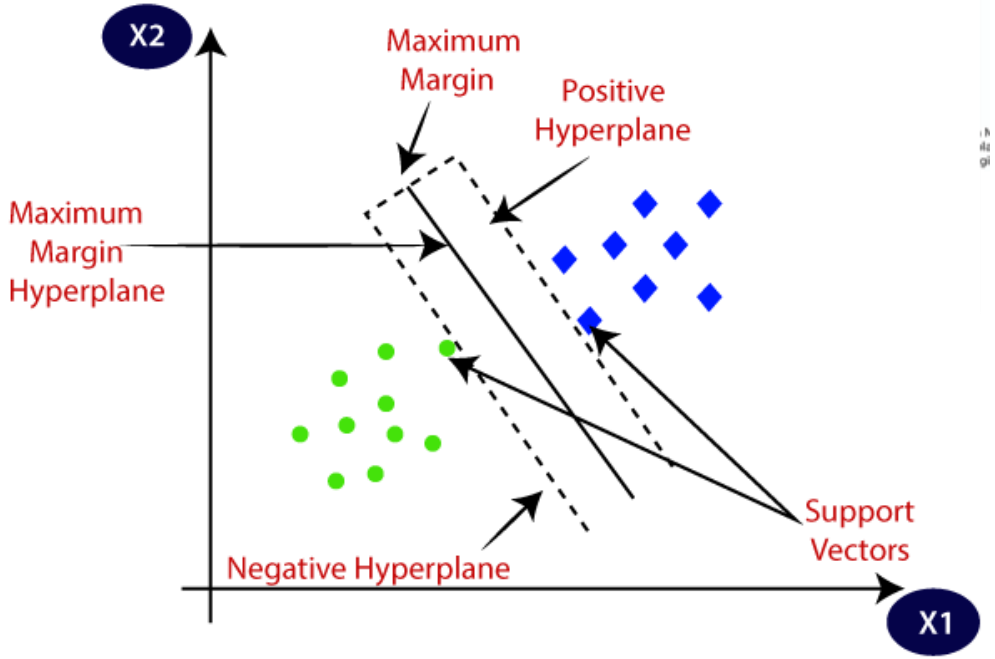
- Find hyperplane maximizes the margin => B1 is better than B2

Support Vector Machine



$$f(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x} + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x} + b \leq -1 \end{cases}$$

$$\text{Margin} = \frac{2}{\| \vec{w} \|}$$



- SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:

Hyperplane and Support Vectors in the SVM

- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.
- The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.
- We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.
- **Support Vectors:**
- The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

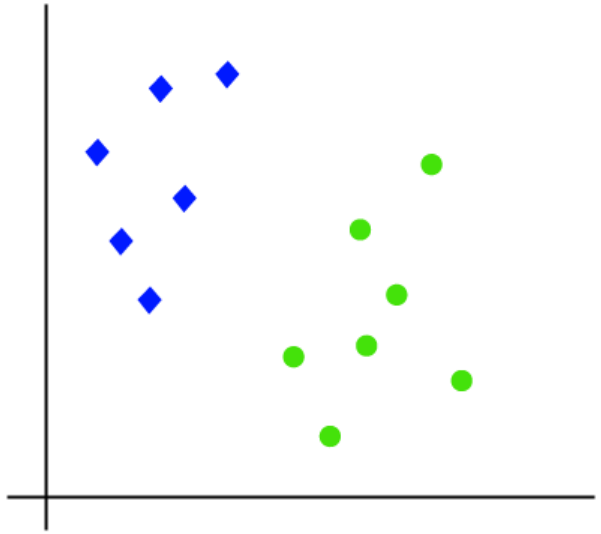
Types of SVM

- **SVM can be of two types:**
  - **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
  - **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

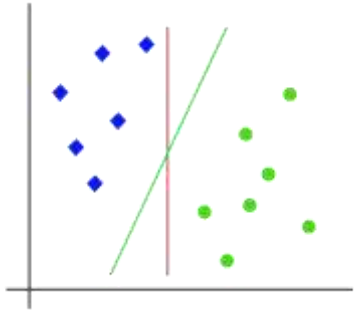
How does SVM works?

Linear SVM:

- The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x1 and x2. We want a classifier that can classify the pair(x1, x2) of coordinates in either green or blue. Consider the below image:

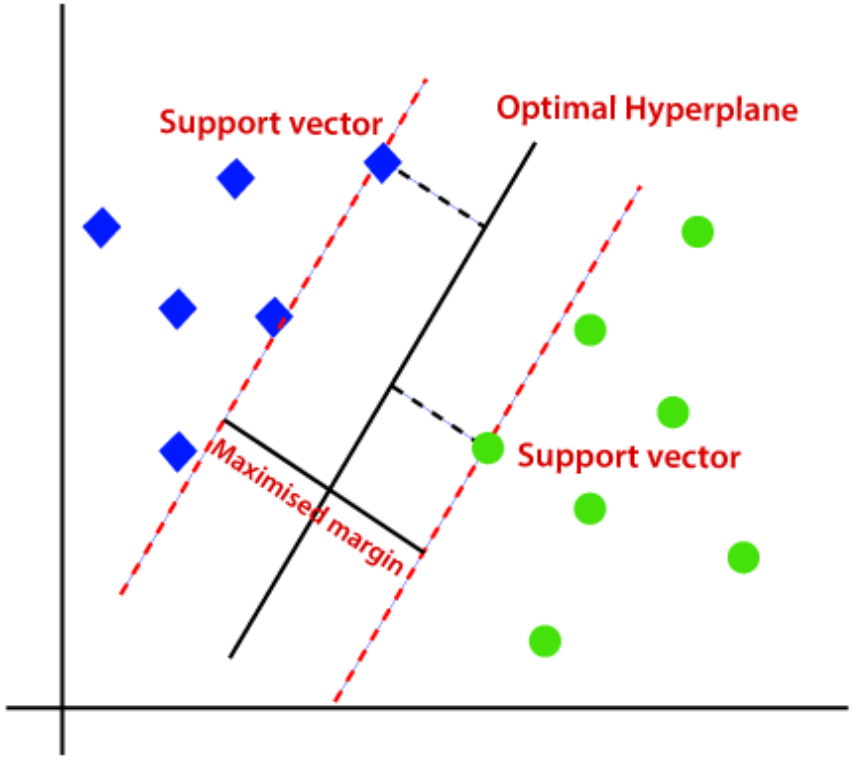


- So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:



- Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.

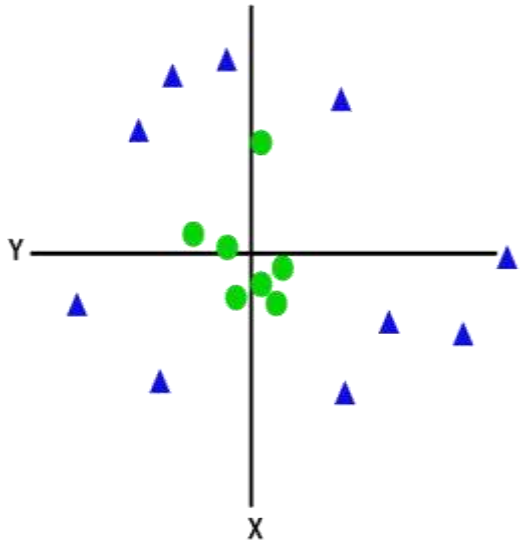
- **Linear SVM**



Distance between the support vectors and the hyperplane should be as far as possible.

Non-Linear SVM

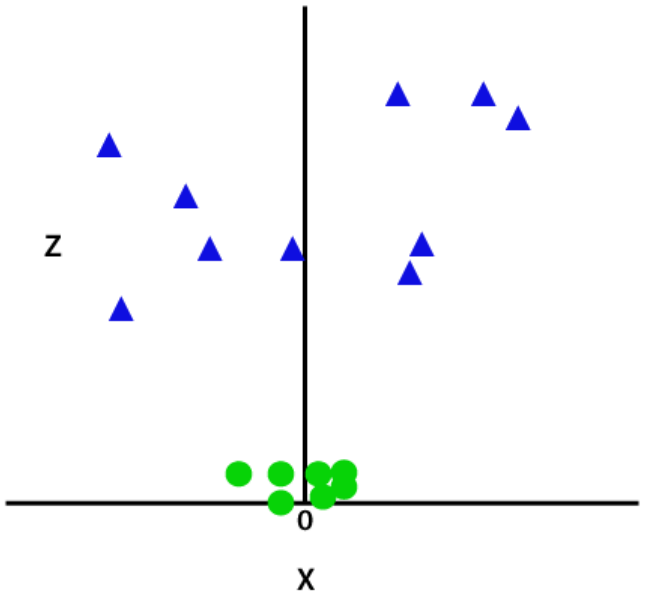
- If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



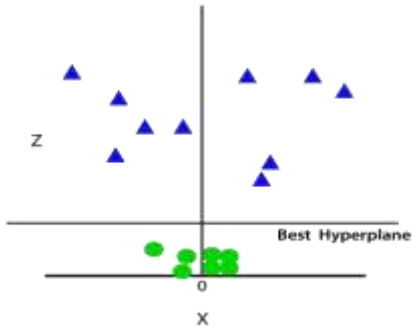
- So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z=x^2 +y^2$$

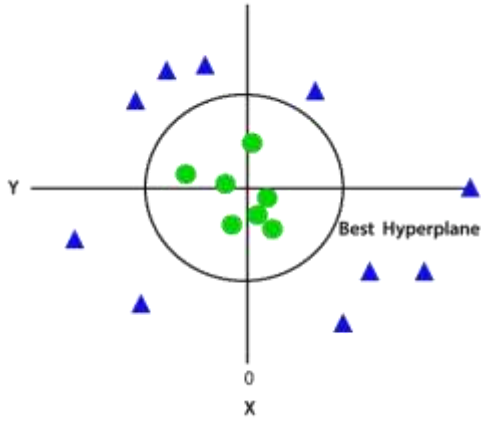
- By adding the third dimension, the sample space will become as below image:



- So now, SVM will divide the datasets into classes in the following way. Consider the below image:



- Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with z=1, then it will become as:



- Hence we get a circumference of radius 1 in case of non-linear data.

Linear SVM

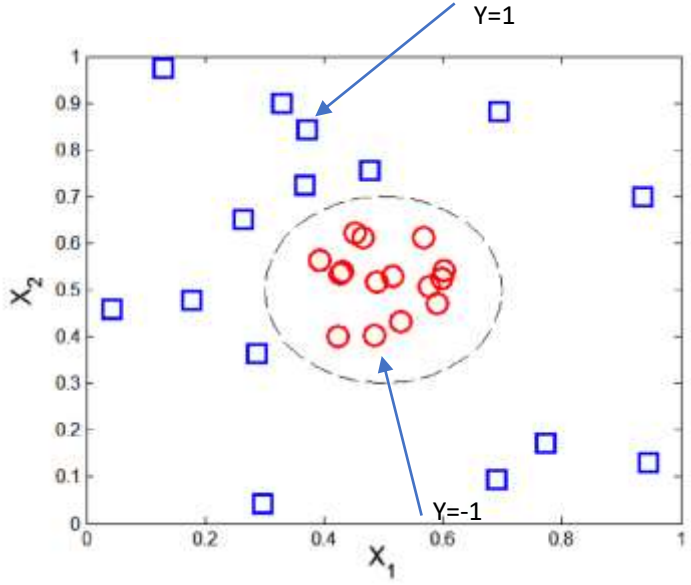
- Linear model:

$$f(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x} + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x} + b \leq -1 \end{cases}$$

- Learning the model is equivalent to determining the values of $\vec{w}$ and b
  - How to find $\vec{w}$ and b from training data?

Nonlinear Support Vector Machine

- What if decision boundary is not linear?



Euclidean Distance

$$y(x_1, x_2) = \begin{cases} 1 & \text{if } \sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} > 0.2 \\ -1 & \text{otherwise} \end{cases}$$

Characteristics of SVM

- The learning problem is formulated as a convex optimization problem
  - Efficient algorithms are available to find the global minima
  - Many of the other methods use greedy approaches and find locally optimal solutions
  - High computational complexity for building the model

- Robust to noise
- Overfitting is handled by maximizing the margin of the decision boundary,
- SVM can handle irrelevant and redundant attributes better than many other techniques
- The user needs to provide the type of kernel function and cost function
- Difficult to handle missing values

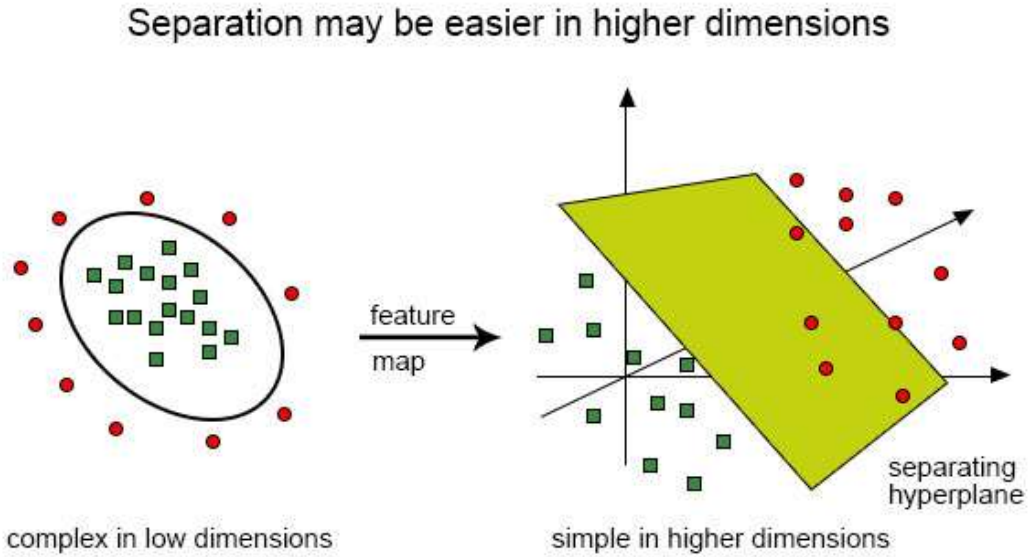
What is Kernel method in machine learning?

- Kernels or kernel methods (also called Kernel functions) are sets of different types of algorithms that are being used for pattern analysis.
- They are used to solve a non-linear problem by using a linear classifier. Kernels Methods are employed in SVM (Support Vector Machines) which are used in classification and regression problems.
- The SVM uses what is called a “Kernel Trick” where the data is transformed and an optimal boundary is found for the possible outputs.
- You can break SVM strategy down into two steps:
  - First, the data is projected implicitly onto a high-dimensional space through the kernel trick.
  - The second step involves applying a linear classifier to the projected data.

Kernel Method

- The Kernel function will usually convert the training set of data so that a non-linear decision surface can be transformed to a linear equation in a higher number of dimension spaces.
- Essentially, it gives back the inner product between two points in a standard feature dimension.
- Kernel functions are applied to every data instance for the purpose of mapping the original nonlinear observations into a higher-dimensional space.
- These observations become separable in the higher-dimensional space.

The Need for Kernel Method and its Working



It is very difficult to solve this classification using a linear classifier as there is no good linear line that should be able to classify the red and the green dots as the points are randomly distributed.

- Here comes the use of kernel function which takes the points to higher dimensions, solves the problem over there and returns the output.
- Think of this in this way, we can see that the green dots are enclosed in some perimeter area while the red one lies outside it, likewise, there could be other scenarios where green dots might be distributed in a trapezoid-shaped area.
- So what we do is to convert the two-dimensional plane which was first classified by one-dimensional hyperplane (“or a straight line”) to the three-dimensional area and here our classifier i.e. hyperplane will not be a straight line but a two-dimensional plane which will cut the area.

- In order to get a mathematical understanding of kernel, let us understand the Lili Jiang's equation of kernel which is:

$$K(x, y)=<f(x), f(y)>$$

where,

- K is the kernel function,
- X and Y are the dimensional inputs,
- f is the map from n-dimensional to m-dimensional space and,
- < x, y > is the dot product.

Illustration with the help of an example.

Let us say that we have two points, x= (2, 3, 4) and y= (3, 4, 5)

As we have seen, K(x, y) = < f(x), f(y) >.

Let us first calculate < f(x), f(y) >

f(x)=(x1x1, x1x2, x1x3, x2x1, x2x2, x2x3, x3x1, x3x2, x3x3)

f(y)=(y1y1, y1y2, y1y3, y2y1, y2y2, y2y3, y3y1, y3y2, y3y3)

so,

f(2, 3, 4)=(4, 6, 8, 6, 9, 12, 8, 12, 16)and

f(3 ,4, 5)=(9, 12, 15, 12, 16, 20, 15, 20, 25)

so the dot product,

f (x). f (y) = f(2,3,4) . f(3,4,5)=(36 + 72 + 120 + 72 +144 + 240 + 120 + 240 + 400)=1444

And, K(x, y) = (2*3 + 3*4 + 4*5) ^2=(6 + 12 + 20)^2=38*38=1444.

This as we find out, f(x).f(y) and K(x, y) give us the same result, but the former method required a lot of calculations(because of projecting 3 dimensions into 9 dimensions) while using the kernel, it was much easier.

What are the types of Kernel methods in SVM models?

Support vector machines use various kinds of kernel methods. Here are a few of them:

1. Liner Kernel

Let us say that we have two vectors with name x1 and Y1, then the linear kernel is defined by the dot product of these two vectors:

$$K(x1, x2) = x1 \cdot x2$$

2. Polynomial Kernel

A polynomial kernel is defined by the following equation:

$$K(x1, x2) = (x1 \cdot x2 + 1)^d$$

Where, d is the degree of the polynomial and x1 and x2 are vectors

3. Gaussian Kernel

$$k(\mathbf{x}_1, \mathbf{x}_n) = e^{-\gamma||\mathbf{x}_1 - \mathbf{x}_n||^2}$$

4. Exponential Kernel

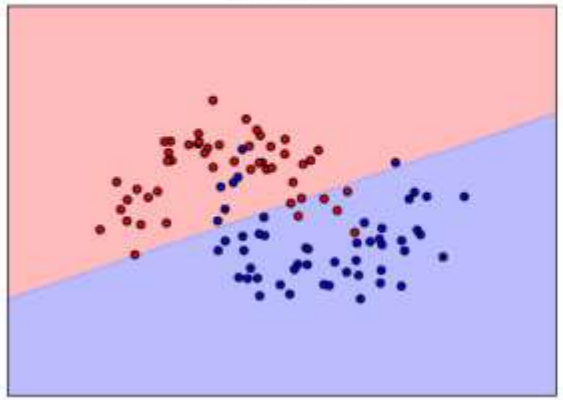
$$k(x, y) = \exp\left(-\frac{||x - y||}{2\sigma^2}\right)$$

5. Laplacian Kernel

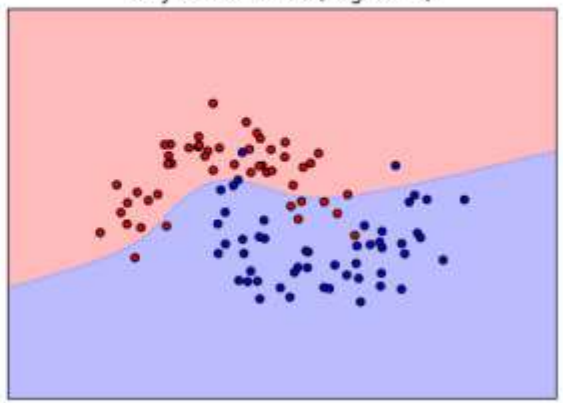
$$k(x, y) = \exp\left(-\frac{\|x - y\|}{\sigma}\right)$$

SVM Decision Boundaries with Different Kernels

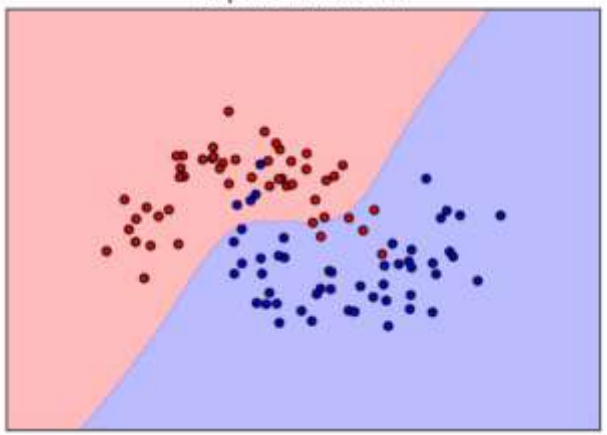
Linear Kernel



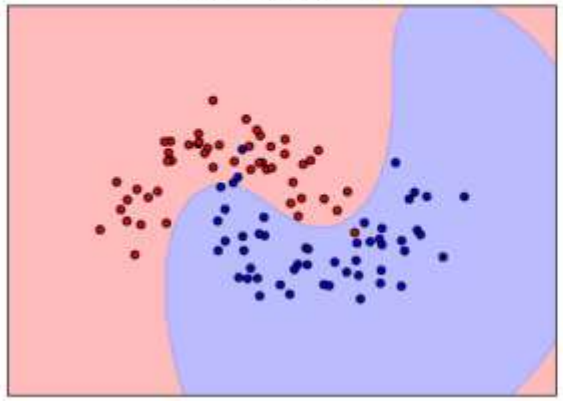
Polynomial Kernel (degree=3)



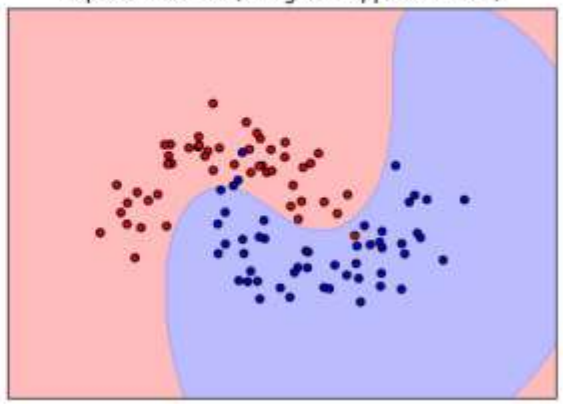
Exponential Kernel



Gaussian (RBF) Kernel



Laplacian Kernel (using RBF approximation)



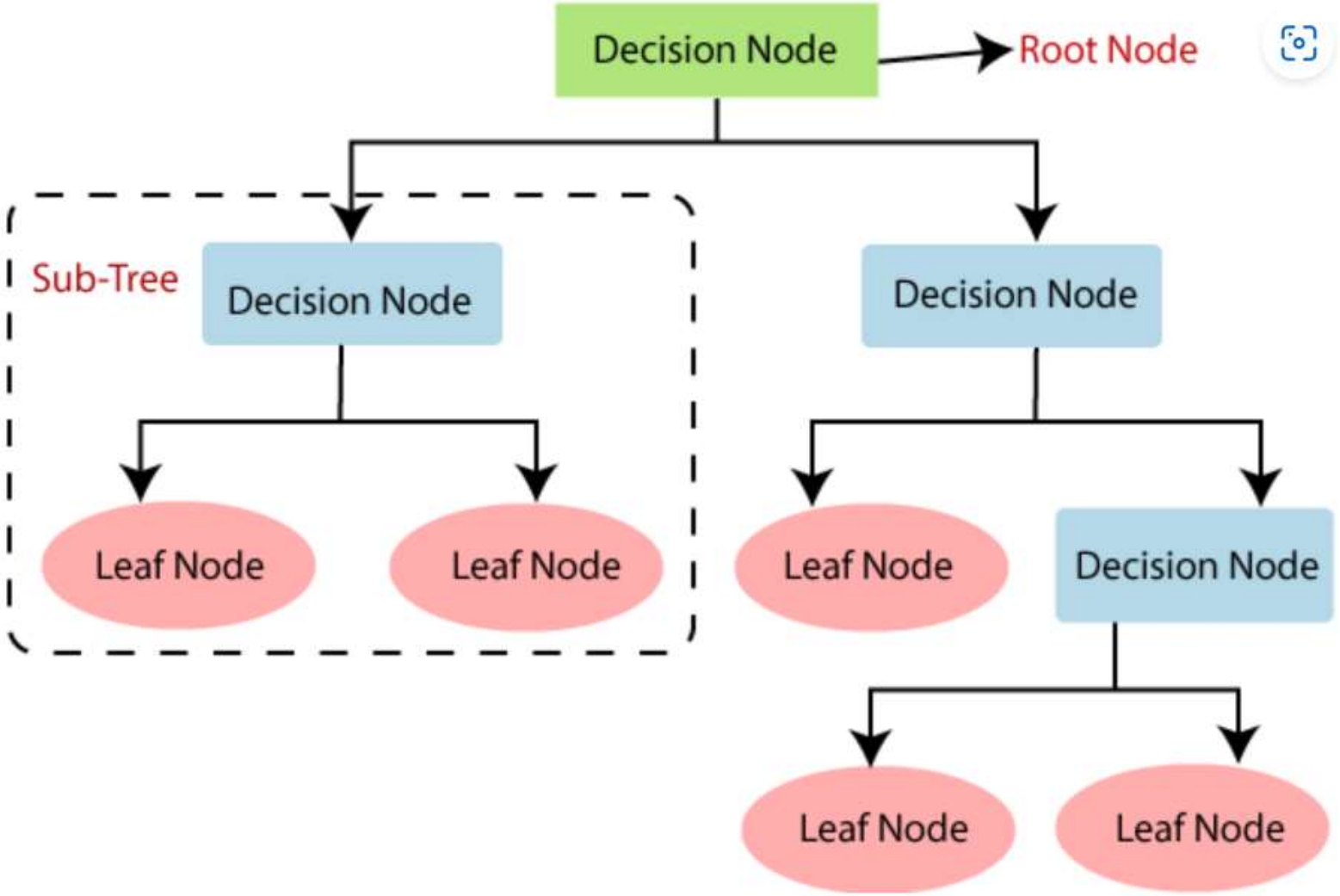
Decision Trees

- Supervised learning technique
  - Used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.
- Tree-structured classifier
  - Internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.

Decision Trees

- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.

Decision Trees



Decision Trees

1.Root Node: It represents the entire population or sample and this further gets divided into two or more homogeneous sets.

2.Splitting: It is a process of dividing a node into two or more sub-nodes.

3.Decision Node: When a sub-node splits into further sub-nodes, then it is called the decision node.

4.Leaf / Terminal Node: Nodes do not split is called Leaf or Terminal node.

5.Pruning: When we remove sub-nodes of a decision node, this process is called pruning. You can say the opposite process of splitting.

6.Branch / Sub-Tree: A subsection of the entire tree is called branch or sub-tree.

7.Parent and Child Node: A node, which is divided into sub-nodes is called a parent node of sub-nodes whereas sub-nodes are the child of a parent node.

Decision Trees

Types of Decision Trees

Types of decision trees are based on the type of target variable we have. It can be of two types:

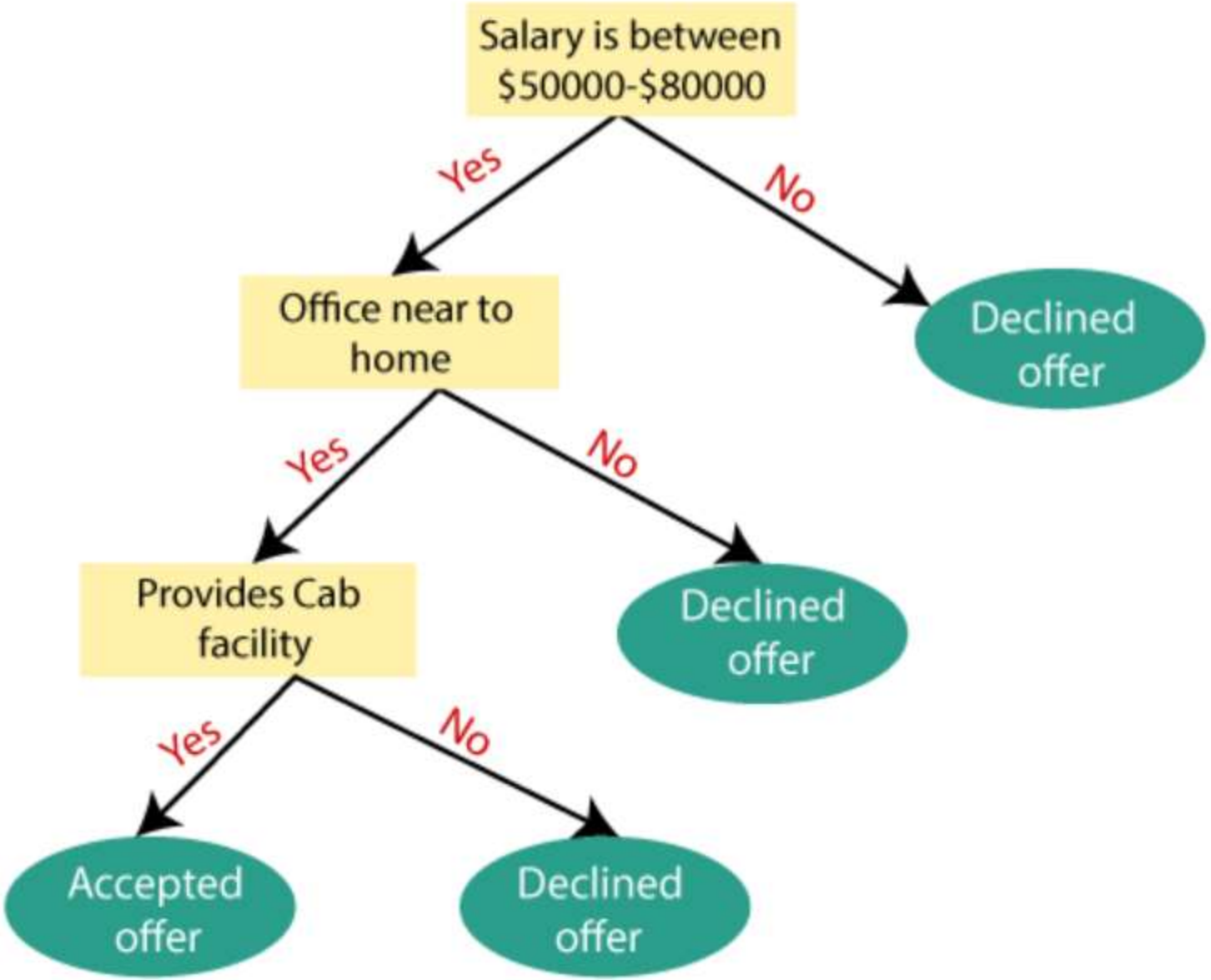
1. **Categorical Variable Decision Tree:** Decision Tree which has a categorical target variable then it called a **Categorical variable decision tree.**
2. **Continuous Variable Decision Tree:** Decision Tree has a continuous target variable then it is called **Continuous Variable Decision Tree.**

Decision Trees

How does the Decision Tree algorithm Work?

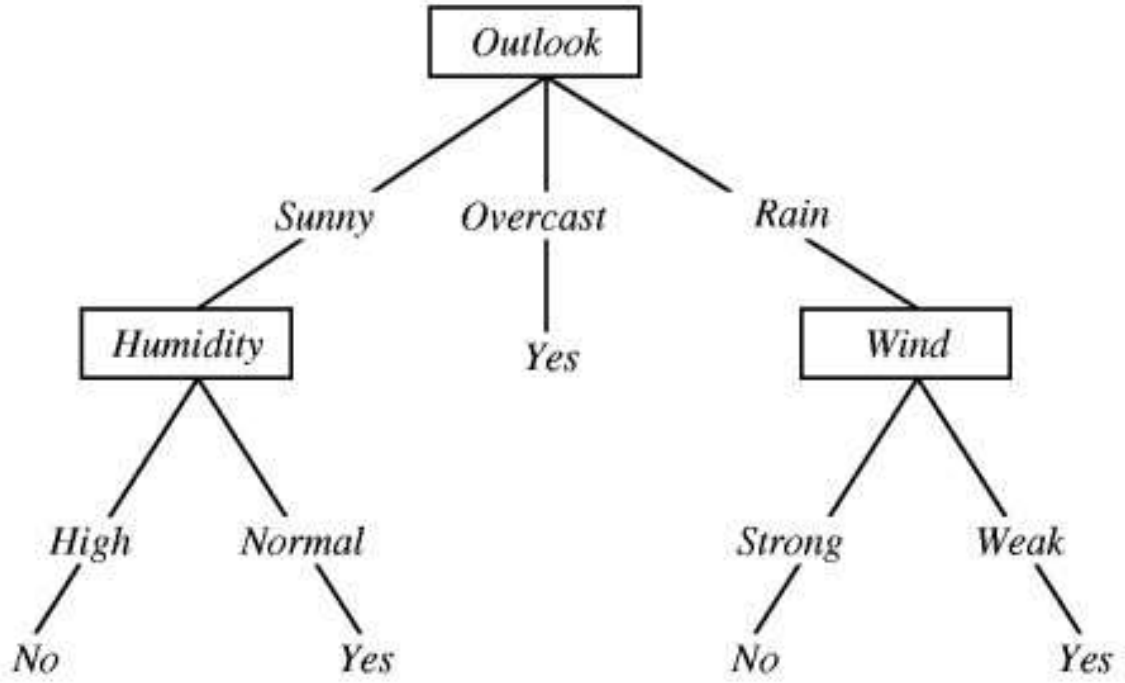
- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Decision Trees



A Decision Tree for the concept *PlayTennis*

- This tree classifies Saturday mornings according to whether or not they are suitable for playing tennis.



Training Dataset

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Algorithms used in Decision Trees:

ID3 → (extension of D3)

C4.5 → (successor of ID3)

CART → (Classification And Regression Tree)

CHAID → (Chi-square automatic interaction detection Performs multi-level splits when computing classification trees)

MARS → (multivariate adaptive regression splines)

Steps in ID3 algorithm:

Steps in ID3 algorithm:

1. It begins with the original set S as the root node.
2. On each iteration of the algorithm, it iterates through the very unused attribute of the set S and calculates **Entropy(H)** and **Information gain(IG)** of this attribute.
3. It then selects the attribute which has the smallest Entropy or Largest Information gain.
4. The set S is then split by the selected attribute to produce a subset of the data.
5. The algorithm continues to recur on each subset, considering only attributes never selected before.

Attribute Selection Measures

- If the dataset consists of N attributes then deciding which attribute to place at the root or at different levels of the tree as internal nodes is a complicated step.
- By just randomly selecting any node to be the root can't solve the issue.
- If we follow a random approach, it may give us bad results with low accuracy.

Attribute Selection Measures

- Entropy
- Information gain
- Gini index
- Gain Ratio
- Reduction in Variance
- Chi-Square

Attribute Selection Measures- Entropy

- Measure of the randomness in the information being processed.
- The higher the entropy, the harder it is to draw any conclusions from that information.
- Flipping a coin is an example of an action that provides information that is random.

Attribute Selection Measures- Entropy

- Entropy for an attribute is represented as:

$$E = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

- p_i is the probability of randomly selecting an example in class i.

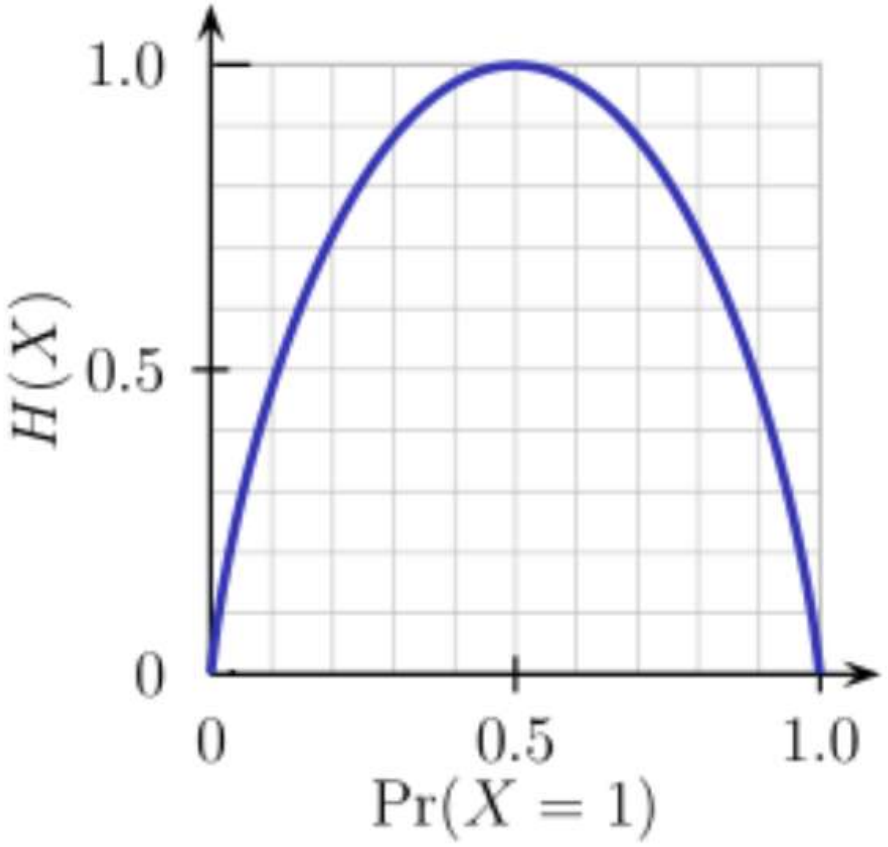
Play Golf	
Yes	No
9	5



Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

Attribute Selection Measures- Entropy

- Entropy H(X) is zero when the probability is either 0 or 1.
- The Entropy is maximum when the probability is 0.5 because it projects perfect randomness in the data and there is no chance if perfectly determining the outcome.



ID3 follows the rule — A branch with an entropy of zero is a leaf node and A brach with entropy more than zero needs further splitting.

Example

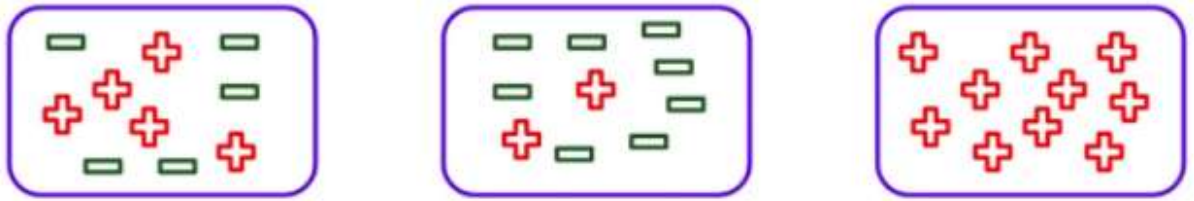
To illustrate, suppose S is a collection of 14 examples of some boolean concept, including 9 positive and 5 negative examples (we adopt the notation [9+, 5−] to summarize such a sample of data). Then the entropy of S relative to this boolean classification is

$$Entropy([9+, 5-]) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$$
$$= 0.940 \qquad (3.2)$$

Notice that the entropy is 0 if all members of S belong to the same class. For example, if all members are positive ($p_\oplus = 1$), then $p_\ominus$ is 0, and $Entropy(S) = -1 \cdot \log_2(1) - 0 \cdot \log_2 0 = -1 \cdot 0 - 0 \cdot \log_2 0 = 0$. Note the entropy is 1 when the collection contains an equal number of positive and negative examples. If the collection contains unequal numbers of positive and negative examples, the

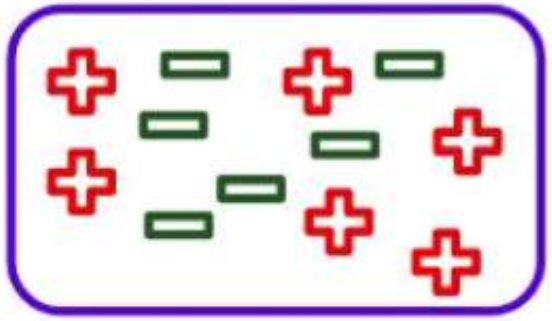
entropy is between 0 and 1.

Node purity



Node 1 Node 2 Node 3

Entropy calculation



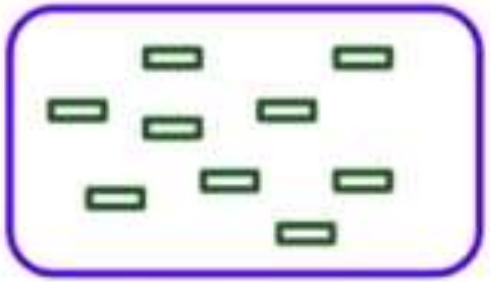
% Play = 0.50

% Not play = 0.50

- Here the percentage of students who play cricket is 0.5 and the percentage of students who do not play cricket is of course also 0.5.
- Since the log of 0.5 bases two is -1, the entropy for this node will be 1

$$\text{Entropy} = -(0.5) * \log_2(0.5) - (0.5) * \log_2(0.5)$$

Entropy calculation in a pure node



% Play = 0

% Not play = 1

Entropy = - (0) * log2(0) - (1) * log2(1)

= 0

Entropy is zero here

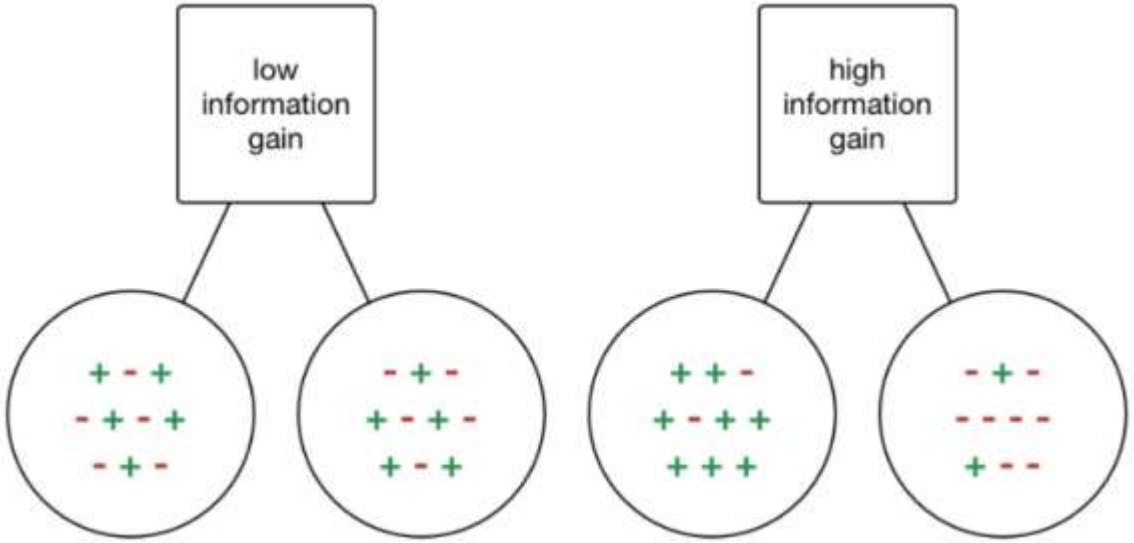
Lower entropy means more pure node and higher entropy means less pure nodes.

Attribute Selection Measures- Information Gain

- Information gain or **IG** is a statistical property that measures how well a given attribute separates the training examples according to their target classification.

- Constructing a decision tree is all about finding an attribute that returns the highest information gain and the smallest entropy.

Information Gain



- Information gain is a decrease in entropy.
- It computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values.
- ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.

Information gain as entropy reduction

- a measure of the effectiveness of an attribute in classifying the training data is called ***information gain,***
- ***This is the*** expected reduction in entropy caused by partitioning the examples according to this attribute.
- The information gain, ***Gain(S, A) of an attribute A,***

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where $Values(A)$ is the set of all possible values for attribute A, and S_v subset of S for which attribute A has value v (i.e., $S_v = \{s \in S | A(s) = v\}$)

Example

For example, suppose S is a collection of training-example days described by attributes including *Wind*, which can have the values *Weak* or *Strong*. As before, assume S is a collection containing 14 examples, [9+, 5−]. Of these 14 examples, suppose 6 of the positive and 2 of the negative examples have $Wind = Weak$, and the remainder have $Wind = Strong$. The information gain due to sorting the original 14 examples by the attribute *Wind* may then be calculated as

$$
\begin{aligned}
Values(Wind) &= Weak, Strong \\
S &= [9+, 5-] \\
S_{Weak} &\leftarrow [6+, 2-] \\
S_{Strong} &\leftarrow [3+, 3-] \\
Gain(S, Wind) &= Entropy(S) - \sum_{v \in \{Weak, Strong\}} \frac{|S_v|}{|S|} Entropy(S_v) \\
&= Entropy(S) - (8/14) Entropy(S_{Weak}) \\
&\quad - (6/14) Entropy(S_{Strong}) \\
&= 0.940 - (8/14)0.811 - (6/14)1.00 \\
&= 0.048
\end{aligned}
$$

Humidity	Wind	Play
High	Week	Yes
High	Week	Yes
Normal	Week	Yes
Normal	Strong	Yes
Normal	Strong	Yes
High	Week	No
High	Strong	No
High	Week	Yes
Normal	Week	Yes
Normal	Week	Yes
Normal	Strong	Yes
High	Week	No
High	Strong	No
Normal	Strong	No

The target variable, "PlayTennis," has two possible values: "Yes" and "No."

From the dataset:

- Total number of samples = 14
- Number of "Yes" = 9
- Number of "No" = 5

The formula for entropy is:

$$\text{Entropy}(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

where p_+ is the probability of "Yes" and p_- is the probability of "No."

Calculating Probabilities

$$p_+ = \frac{9}{14} = 0.6429$$

$$p_- = \frac{5}{14} = 0.3571$$

Calculating Total Entropy

$$\text{Entropy}(S) = -(0.6429 \cdot \log_2(0.6429)) - (0.3571 \cdot \log_2(0.3571))$$

$$\approx -(0.6429 \cdot 0.6439) - (0.3571 \cdot 1.4854)$$

$$\approx -0.4134 - 0.5269 = 0.9403$$

So, the **Total Entropy** of "PlayTennis" is approximately **0.94**.

- "High" Humidity has 7 samples:
  - 3 "Yes"
  - 4 "No"

Probability of "Yes" and "No" for High Humidity

$$p_{\text{High, Yes}} = \frac{3}{7} = 0.4286$$

$$p_{\text{High, No}} = \frac{4}{7} = 0.5714$$

Entropy for "Humidity = High"

$$\text{Entropy}(S_{\text{High}}) = -(0.4286 \cdot \log_2(0.4286)) - (0.5714 \cdot \log_2(0.5714))$$

$$\approx -(0.4286 \cdot 1.2224) - (0.5714 \cdot 0.8074)$$

$$\approx -0.5232 - 0.4616 = 0.985$$

- "Normal" Humidity has 7 samples:
  - 6 "Yes"
  - 1 "No"

Probability of "Yes" and "No" for Normal Humidity

$$p_{\text{Normal, Yes}} = \frac{6}{7} = 0.8571$$

$$p_{\text{Normal, No}} = \frac{1}{7} = 0.1429$$

Entropy for "Humidity = Normal"

$$\text{Entropy}(S_{\text{Normal}}) = -(0.8571 \cdot \log_2(0.8571)) - (0.1429 \cdot \log_2(0.1429))$$

$$\approx -(0.8571 \cdot 0.2224) - (0.1429 \cdot 2.8074)$$

$$\approx -0.1905 - 0.4011 = 0.5917$$

$$\text{Weighted Entropy(Humidity)} = \left(\frac{7}{14}\right) \cdot 0.985 + \left(\frac{7}{14}\right) \cdot 0.5917$$

$$= 0.5 \cdot 0.985 + 0.5 \cdot 0.5917$$

$$= 0.4925 + 0.2959 = 0.7884$$

$$\text{Information Gain(Humidity)} = \text{Total Entropy} - \text{Weighted Entropy(Humidity)}$$

$$= 0.9403 - 0.7884 = 0.1519$$

So, the **Information Gain for "Humidity"** is approximately **0.15**.

- "Weak" Wind has 8 samples:
  - 6 "Yes"
  - 2 "No"

Probability of "Yes" and "No" for Weak Wind

$$p_{\text{Weak, Yes}} = \frac{6}{8} = 0.75$$

$$p_{\text{Weak, No}} = \frac{2}{8} = 0.25$$

Entropy for "Wind = Weak"

$$\text{Entropy}(S_{\text{Weak}}) = -(0.75 \cdot \log_2(0.75)) - (0.25 \cdot \log_2(0.25))$$

$$\approx -(0.75 \cdot 0.4150) - (0.25 \cdot 2.0)$$

$$\approx -0.3113 - 0.5 = 0.8113$$

- "Strong" Wind has 6 samples:
  - 3 "Yes"
  - 3 "No"

Probability of "Yes" and "No" for Strong Wind

$$p_{\text{Strong, Yes}} = \frac{3}{6} = 0.5$$

$$p_{\text{Strong, No}} = \frac{3}{6} = 0.5$$

Entropy for "Wind = Strong"

$$\text{Entropy}(S_{\text{Strong}}) = -(0.5 \cdot \log_2(0.5)) - (0.5 \cdot \log_2(0.5))$$

$$= -(0.5 \cdot 1) - (0.5 \cdot 1)$$

$$= -0.5 - 0.5 = 1.0$$

$$\text{Weighted Entropy(Wind)} = \left(\frac{8}{14}\right) \cdot 0.8113 + \left(\frac{6}{14}\right) \cdot 1.0$$

$$= 0.5714 \cdot 0.8113 + 0.4286 \cdot 1.0$$

$$= 0.4639 + 0.4286 = 0.8925$$

$$\text{Information Gain(Wind)} = \text{Total Entropy} - \text{Weighted Entropy(Wind)}$$

$$= 0.9403 - 0.8925 = 0.0478$$

So, the **Information Gain for "Wind"** is approximately **0.048**.

Which attribute is the best classifier?

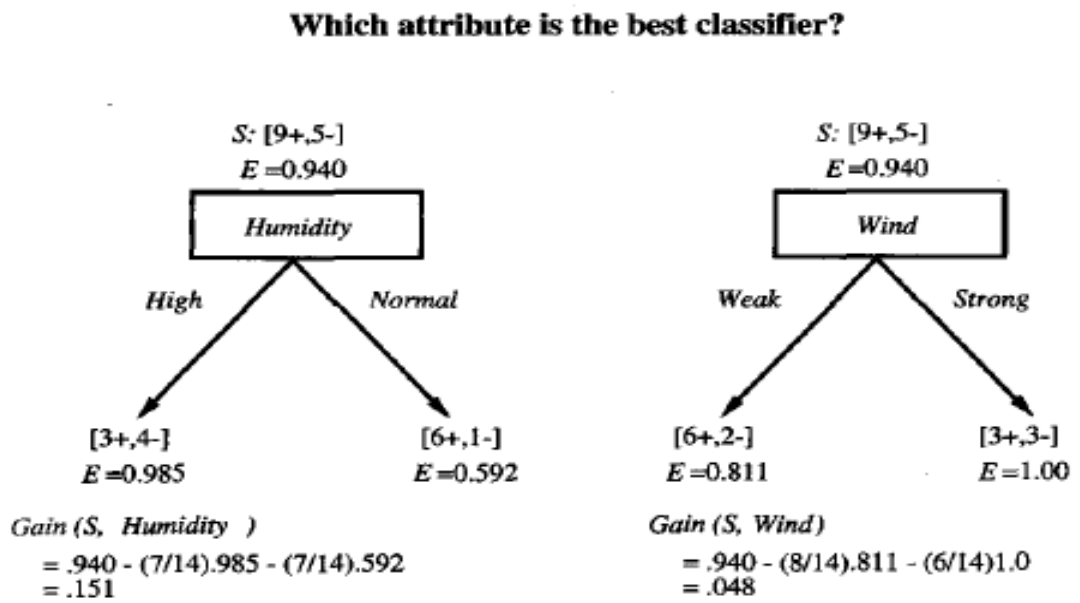


FIGURE 3.3
Humidity provides greater information gain than *Wind*, relative to the target classification. Here, E stands for entropy and S for the original collection of examples. Given an initial collection S of 9 positive and 5 negative examples, [9+, 5−], sorting these by their *Humidity* produces collections of [3+, 4−] (*Humidity* = *High*) and [6+, 1−] (*Humidity* = *Normal*). The information gained by this partitioning is .151, compared to a gain of only .048 for the attribute *Wind*.

Gini Index

- The Gini index is a measure of impurity or diversity used in decision tree algorithms, particularly in classification tasks. It helps the algorithm determine the optimal split at each node of the tree during the training process.
- The Gini index ranges from 0 to 1, where 0 indicates that the node is pure (all instances belong to the same class), and 1 indicates maximum impurity (the classes are evenly distributed).
- Used in CART

$$\text{Gini} = 1 - \sum_{i=1}^{n} p_i^2$$

- p_i is the probability of an instance belonging to class i,
- n is the number of classes.

Decision Tree

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm.**
- For more class labels, the computational complexity of the decision tree may increase.

How to avoid/counter Overfitting in Decision Trees?

- Building trees that “adapt too much” to the training examples may lead to “overfitting”.
- Way to remove overfitting:

  Pruning Decision Trees

Pruning Decision Trees

- The splitting process results in fully grown trees until the stopping criteria are reached. But, the fully grown tree is likely to overfit the data, leading to poor accuracy on unseen data.

Pruning Decision Trees

Pruning: Getting an Optimal Decision tree

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree **pruning** technology used:

- **Cost Complexity Pruning**
- **Reduced Error Pruning.**

Pruning Decision Trees

- In pruning, you trim off the branches of the tree, i.e., remove the decision nodes starting from the leaf node such that the overall accuracy is not disturbed.
- This is done by segregating the actual training set into two sets: training data set, D and validation data set, V.
- Prepare the decision tree using the segregated training data set, D.
- Then continue trimming the tree accordingly to optimize the accuracy of the validation data set, V.

Pruning Decision Trees

